# Appendix G — GitLab master study guide — everything Karan can ask from your resume

Source: `GITLAB_MASTER_STUDY_GUIDE.md`

## GitLab Master Study Guide — Everything Karan Bhatia Can Ask From Your Resume

**Role:** Senior Software Engineer, Agentic SDLC (Duo Agent Platform), GitLab. **Interviewer:** Karan Bhatia — VP Engineering & Engineering Fellow, GitLab. **Interview:** Friday 2026-10-09. Prepared 2026-10-02.

This is the single consolidated document. It does three things:

1. **Condenses** what already exists in the other prep docs (and says exactly where the full depth lives, so nothing is silently dropped).
2. **Teaches, from the ground up,** every topic on your resume that the other docs did not yet cover in depth: LangGraph internals, MCP as a protocol, the RAG variants by name, vector databases, reranking/chunking/embeddings, model routing and token economics, AWS Lambda/ECS/API Gateway, Docker and Kubernetes, Spring Boot / REST / GraphQL / microservices, OpenTelemetry, and your Disney+ Hotstar, MakeMyTrip and AutoRABIT work.
3. **Maps every line of the resume** to a section, a status flag, and a question bank.

Source documents referenced throughout (abbreviations used in cross-references):

| Abbrev. | File | What it holds |
|---|---|---|
| **DD** | `GITLAB_KARAN_BHATIA_DEEP_DIVE.md` | The 80-page reference: GitLab architecture, resume defense, STAR stories, infra (§14) |
| **SG-GL** | `GITLAB_STUDY_GUIDE.md` | Taught sections: permission gate, funnel, persistence, compaction, Data Cloud |
| **QB** | `GITLAB_INTERVIEW_QUESTION_BANK.md` | Q&A with READY / PENDING / HONEST GAP flags |
| **WP** | `GITLAB_WEEK_PLAN.md` | The Oct 2–9 schedule |
| **PROBE** | `~/Desktop/salesforce_architecture_probe.md` | Real Agentforce v2 architecture + office-Claude prompts |
| **OCQ** | `OFFICE_CLAUDE_QUESTIONS.md` | Infra / deployment / Data Cloud probe prompts (A–F) |
| **AS** | `ANIRUDDHA_SINGH_DEEP_DIVE.md` | Agentforce internals §1, vendor-neutral trust model §2 |
| **SG** | `study_guide_enterprise_system_v2.md` | Vendor-neutral enterprise agent framework |
| **HLD** | `~/Downloads/High-Level_Design_Agentforce_for_Setup_v2.pdf` | Primary source (Jan 16 2026) |
| **ART** | `~/Downloads/Agent_Runtime_Deep_Dive.pdf` | Agent-runtime teaching companion |
| **RAG-S** | `RAG_SIMPLE.md` | Plain-language RAG basics |
| **POST** | `pdf/medium_post.md` | Your own "From Prompt to Platform" post |

# Part 0 — How to use this document

## 0.1 Status-flag legend (same convention as QB)

- **READY** — grounded in a real source (code you wrote, a primary design doc, a verified public fact). Safe to answer with confidence today.
- **PENDING** — the mechanism is known, but a specific number, config value, or personal-share claim needs verification (office-Claude via OCQ/PROBE, or re-running your own code) before you state it as fact in the room.
- **HONEST GAP** — explicitly not built / not operated by you. Say so plainly, then reason from first principles. Never improvise a system you did not run.

A fourth label from DD §14 applies to every infrastructure question: **OWN / PATTERN / THIN** — state your role in one clause before answering ("my team ran the cluster; I owned the service's deployment config and alerts").

**One correction relative to QB §6:** QB lists "session persistence / crash recovery" as an HONEST GAP. That is true for Salesforce, **not** for ResolveFlow. ResolveFlow ships durable pause/resume with `interrupt()` + `AsyncPostgresSaver` (verified in the repo). Use the narrower framing from SG-GL §3: "*Shipped in ResolveFlow; not something I shipped at Salesforce.*" Compaction remains a genuine HONEST GAP (SG-GL §4 has the project to close it).

## 0.2 The interviewer in one paragraph

Karan Bhatia joined GitLab ~Sep 2026 as VP Engineering & Engineering Fellow, so on Oct 9 he is about two months into GitLab's codebase himself. Before that: New Relic 2022–2026 (Principal → Senior Principal Architect → VP Eng & Technical Fellow), platform architecture across Security, Billing, Identity & Access Management and Growth, with a self-described specialty in "scaling multi-tenanted systems"; Microsoft Azure Active Directory 2016–2022 (5.5 years of identity and access management at cloud scale); Accenture Federal (operated CI and AWS automation). His public writing ("Observability in a distributed world," New Relic, Aug 2024) is short, concrete and code-first: propagate `traceId`, `upstreamApplication`, `userId`, `tenantId` across every hop, then find the root cause by following one `traceId` with a query. He championed the New Relic × GitHub Copilot closed loop (regression detected → issue filed → coding agent drafts fix → PR for human review → post-merge validation), which is this job in miniature. Full profile: DD §0.1.

**How a Fellow probes (DD §0.1 table, condensed):** he checks where your mechanism *stops working*; asks *which token, minted by whom, with what audience*; asks *what's on the span and what's the query*; asks *of how many, what interval, what baseline* for every number; designs with you and then *removes an assumption*; and evaluates how you reason when you don't know. **Bluffing is the one unrecoverable error.**

## 0.3 Your three bridges to him (condensed from DD §0.2 / QB §1)

1. **Identity & authorization propagation (his Azure AD years).** "An agent never holds authority of its own — it borrows the caller's, narrowed." Salesforce: privileged actions run under the requesting user's own permissions; tenant-scoped `orgJWT` via token exchange before retrieval. ResolveFlow: per-visitor GitHub OAuth, server-side `state`, `thread_owners` BOLA fix, `execute()` refuses without approval. GitLab: composite identity = service account ∩ human, most restrictive wins. **Volunteer your weak spot first** (DD §2.4 F4): `public_repo` scope is broad, token stored plaintext, shared-token fallback — fix is a GitHub App with fine-grained, short-lived installation tokens and envelope encryption.
2. **Observability as agent correctness (his current passion).** Offline evals = pre-production SLO check; online evals = production SLI; production failures become eval cases. Plus his correlation-attribute pattern applied to agent runs, persisted *in the checkpoint* so the trace survives the durable pause (Part 5 here; DD §4.8).

3. **Multi-tenancy at scale (his specialty).** "`tenant_id` is not merely a database column. It is security context" (POST §23). Pre-filtered retrieval, tenant-scoped cache keys, per-tenant fairness in shared worker pools (DD §5.3).

**Order when he says "tell me about your work":** identity + multi-tenancy first → observability / evals second → the agent architecture as the thing those two make safe.

### 0.4 The four-beat answer shape (ART §12, applied to him)

1. Name the concept plainly. 2. Say what failure it prevents. 3. Point to *your* system.
2. Volunteer the trade-off / where it breaks — and the signal you would watch for it.

### 0.5 How the rest of this doc is organized

| Part | Topic | New or condensed |
|---|---|---|
| 1 | Agentic SDLC & orchestration: LangGraph, MCP, multi-agent, HITL, permission gate | LangGraph + MCP **new, deep**; rest condensed |
| 2 | LLM / RAG / retrieval: RAG variants, vector DBs, rerank/chunk/embed, routing & tokens | **New, deep** |
| 3 | Evaluation, safety & guardrails | Condensed + additions |
| 4 | Distributed systems & deployment: AWS, Docker/K8s, Spring/REST/GraphQL/microservices | **New, deep** + infra audit |
| 5 | Observability: OpenTelemetry + his correlation pattern | OTel **new, deep** |
| 6 | Agentforce for Setup v1 → v2 | Condensed (PROBE / AS §1 for full depth) |
| 7 | ResolveFlow end to end + honesty audit R1–R11 | Condensed |
| 8 | Disney+ Hotstar, MakeMyTrip, AutoRABIT | **New, deep** |
| 9 | Every resume number, one table | New |
| 10 | Master tiered question bank (Foundational → Fellow) | New, spans everything |
| 11 | One-page closing recap + litmus tests | New |

Suggested use with WP: Oct 3 → Parts 1, 7; Oct 4 → Part 4.3 + DD §11 (Ruby); Oct 5 → Parts 2, 6; Oct 6 → Parts 1.1–1.2 again + 3 + 5; Oct 7 → Parts 4, 8; Oct 8 → Part 10 as the mock source; Oct 9 → Part 11 only.

---

## Part 1 — Agentic SDLC & orchestration

### 1.1 LangGraph internals, properly

**Resume lines this defends:** "LangGraph, LangChain"; "agent orchestration patterns"; "durable pause/resume state (LangGraph interrupts)"; "memory & state management"; "multi-agent systems"; ResolveFlow bullets 1–3. **Status: READY** (you wrote the graph; facts below verified against the LangChain docs, Oct 2026).

#### 1.1.1 The idea, slowly

A plain agent is a loop: call the model; if it asks for a tool, run the tool, append the result, call the model again; stop when it answers without a tool request (ART §1). That loop has one property that makes it dangerous for anything with side effects: **the boundary between "thinking" and "acting" is a behavior the prompt requests, not a property the system enforces.** The model decides, turn by turn, whether to act.

LangGraph replaces "a loop the model drives" with **"a state machine you define, inside which the model makes bounded choices."** You declare:

- a **state schema** — the single typed object every step reads and writes;
- **nodes** — plain functions `state -> partial update`;
- **edges** — which node runs next, either fixed or computed from state by your code;
- a **checkpointer** — where every step's state is saved, so the run can pause, crash, resume.

The model can still be inside a node (and can still choose among options you give it), but the legal transitions are code. That is the one-sentence answer to "why LangGraph and not a loop":

> "A loop lets the model wander into any tool at any time. A graph encodes the legal transitions as code, so the agent can't reach the side-effecting node except through the approval node — and every step is checkpointed, so a pause can last hours and survive a restart."

### 1.1.2 Why it exists — what it replaced

| Generation | Shape | Problem |
|---|---|---|
| Raw tool loop | while model wants tools: run tool | No structure, no persistence, unbounded |
| LangChain `AgentExecutor` (2023–24) | A packaged ReAct loop with tools, memory object, `max_iterations` | Opaque control flow, hard to add branches, approvals or parallelism; state was "the chat history"; no first-class durable pause. In LangChain 1.0 (late 2025) `AgentExecutor` and legacy chains moved to the `langchain-classic` package |
| LangGraph | Explicit graph + typed state + checkpointer | You own control flow; durable execution, HITL, streaming, time travel are first-class |
| LangChain 1.0 `create_agent` | A prebuilt agent loop **built on LangGraph**, customized via middleware (`before_model`, `after_model`, `wrap_model_call`, `wrap_tool_call`) | Convenient for a standard ReAct agent; drop to raw LangGraph when you need custom topology (as ResolveFlow does) |

Say it precisely: "*LangChain is the component library — model wrappers, retrievers, text splitters, structured output. LangGraph is the runtime — state, control flow, persistence. ResolveFlow uses LangChain for `ChatOpenAI.with_structured_output`, `OpenAIEmbeddings`, `RecursiveCharacterTextSplitter` and the Pinecone vector store, and LangGraph for the graph, the interrupt and the Postgres checkpointer.*"

### 1.1.3 Mechanics — the execution model (Pregel / supersteps)

LangGraph's runtime is modeled on Google's **Pregel** (bulk-synchronous parallel) graph processing:

1. State is stored in **channels** — one per state key. Each channel has a **reducer** that says how to combine writes.
2. Execution proceeds in **supersteps**. In each superstep, every node whose trigger fired runs — in parallel if more than one is ready. Each node receives a read-only view of state and returns a **partial update** (a dict of only the keys it changed).
3. At the end of the superstep, all updates are applied through their reducers, and a **checkpoint** is written.
4. Edges from the nodes that just ran decide which nodes trigger in the next superstep.
5. The run ends when no node is triggered (you reached `END`), or when a node calls `interrupt()`.

**Reducers — the part people skip.** Default reducer = *overwrite* (last write wins). That's fine for a linear graph. But if two nodes run in the *same* superstep and both write key `x` with no reducer, LangGraph raises `InvalidUpdateError` — it refuses to guess. You declare a reducer with `Annotated`:

```
from typing import Annotated, TypedDict
import operator
from langgraph.graph.message import add_messages

class S(TypedDict):
    findings: Annotated[list[str], operator.add]   # parallel writers append
    messages: Annotated[list, add_messages]        # append; same-id message replaces
    plan: dict                                     # overwrite (default)
```

`add_messages` is the chat reducer: appends new messages, but a message with an existing `id` *replaces* the old one (that's how you edit history).

**ResolveFlow specifically:** `GraphState` (graph/state.py) has **no reducers** — every key is overwrite — and that is correct because the graph is linear per path: no two nodes ever run in the same superstep. The docstring enforces an *append-only by convention* rule ("once fetch_evidence sets `evidence`, nothing downstream should mutate it") which is what makes the state auditable. **The trap he can set:** "add a fan-out that diagnoses with two models in parallel" — then `diagnosis` would need a reducer (or separate keys), or the superstep fails.

### 1.1.4 Mechanics — nodes, edges, conditional edges, `Command`, `Send`

```
g = StateGraph(GraphState)
g.add_node("classify", classify)                      # node = function(state) -> dict
g.add_edge("normalize_evidence", "classify")          # fixed edge
g.add_conditional_edges("classify", should_route_to,  # router = function(state) -> label
    {"deterministic": "await_approval",
     "ai_investigation": "generate_diagnosis",
     "human_review": END})
```

- **Conditional edge:** your *code* reads state and returns a label; the mapping turns the label into the next node. In ResolveFlow all three routers (`should_route_to`, `should_execute`, `should_route_review`) are deterministic Python. **No LLM decides a transition.** That's the resume's "keeping approve/escalate decisions deterministic in application code."
- **`Command(goto=..., update=...)`** — a node can return routing + update together (dynamic routing without a separate conditional edge). `Command(resume=value)` is the input you pass to `invoke` to resume an interrupt; per the docs it is "the **only** `Command` pattern intended as input to `invoke()`/`stream()`". `Command(graph=Command.PARENT, goto=...)` routes from a subgraph to its parent.
- **`Send(node, payload)`** — returned from a conditional edge to **fan out**: N copies of a node run in the same superstep with different inputs; their results merge through reducers. This is LangGraph's map-reduce (DD §1.6 uses it for per-file test generation).
- **Retry policy** — `add_node("plan", plan, retry_policy=RetryPolicy(max_attempts=3))` retries a node on transient exceptions. Only safe on idempotent nodes.

### 1.1.5 Mechanics — checkpointers and threads

A **checkpointer** saves a snapshot of state after every superstep, keyed by a **`thread_id`** you pass in config: `{"configurable": {"thread_id": "..."}}`. Thread = one run's history.

| Checkpointer | Package | Durable across restart? | Use |
|---|---|---|---|
| `InMemorySaver` (older name `MemorySaver`) | `langgraph` | **No** — RAM only | Tests, notebooks. ResolveFlow's `compiled_graph` export in `build.py` |

| Checkpointer | Package | Durable across restart? | Use |
|---|---|---|---|
| `SqliteSaver` / `AsyncSqliteSaver` | `langgraph-checkpoint-sqlite` | Only if the disk survives | Local dev. **Your Story D: Render free tier's disk is ephemeral, so it silently lost paused approvals** |
| `PostgresSaver` / `AsyncPostgresSaver` | `langgraph-checkpoint-postgres` | Yes | Production. ResolveFlow `app/main.py` |

**What a checkpoint contains** (DD §4.4): channel values (the state), channel versions, which nodes have seen which versions (so the runtime knows what's triggered next), **pending writes** (per-task writes from a superstep that partially completed, so finished tasks don't re-run), and metadata (step number, source, `parent_checkpoint_id`). The parent pointer makes a chain, which gives you:

- `graph.get_state(config)` → `StateSnapshot` (values, `next` nodes, config, metadata, tasks including pending interrupts);
- `graph.get_state_history(config)` → the chain, newest first (**time travel**);
- invoke with a specific `checkpoint_id` → **replay/fork** from that point;
- `graph.update_state(config, values)` → write a new checkpoint as if a node produced it (human edits state before resuming).

**Checkpointer interface (your `docs/CHECKPOINTING.md`):** `aget_tuple` (load latest or a given checkpoint), `aput` (write snapshot), `aput_writes` (stage per-task writes), `alist`. The key observation you made: "*there is no separate resume code path in the checkpointer; the only difference is whether a row exists.*" The first `ainvoke` and the resume both call `aget_tuple(thread_id)`.

**Durability modes** (newer LangGraph; passed on the execution call): `"exit"` — persist only when the run exits (finish, error, or interrupt): fastest, but a mid-run crash loses intermediate progress; `"async"` — persist the checkpoint while the next step starts (the default per current behavior): small window of loss on crash; `"sync"` — persist before the next step starts: safest, slowest. **Interview line:** "for a graph whose side-effecting node is guarded by an idempotency key, `async` is fine; if a node's side effect is not idempotent and a crash before the checkpoint would cause a re-run, I'd want `sync` *plus* the idempotency key — the mode narrows the window, the key closes it."

**Checkpointer vs. Store.** Checkpointers are **per-thread, short-term** memory (one run's state). A **Store** (`InMemoryStore`, `PostgresStore`) is **cross-thread, long-term** memory, namespaced (e.g., `("user", user_id, "preferences")`), optionally with semantic search. "Memory & state management" on your resume = know both: ResolveFlow uses the checkpointer; its Pinecone corpus plays the role of long-term knowledge, and SG-GL §4's compaction project would write protected facts to a Store-like location.

### 1.1.6 Mechanics — `interrupt()` and durable execution, precisely

Verified against the LangGraph interrupts doc (Oct 2026):

1. Inside a node, `interrupt(value)` **raises a special exception**. The runtime catches it, saves the checkpoint, and returns to the caller; the result carries `__interrupt__` with your `value` (JSON-serializable only).
2. To resume: `graph.invoke(Command(resume=x), config)` with the **same `thread_id`**, from any process — the state is in Postgres, not memory.
3. **The runtime restarts the entire node from its first line**, not from the line where `interrupt` was called. When execution reaches `interrupt()` again, it *returns `x`* instead of raising.
4. Therefore **everything before `interrupt()` runs twice** and must be deterministic and idempotent. The docs say side effects before an interrupt "must be idempotent."
5. Multiple `interrupt()` calls in one node are matched to resume values **strictly by index** (order). Don't put them inside conditional branches or non-deterministic loops.

6. **Don't wrap `interrupt()` in a bare `try/except`** — you'll swallow the control-flow exception.
7. A checkpointer is mandatory; without one there's nowhere to save the paused state.

**ResolveFlow's version (`await_approval.py`):**

```
def await_approval(state: GraphState) -> dict:
    proposed_action = _build_proposed_action(state)          # runs twice — pure function of state
    approved = interrupt({"classification": state["classification"],
                          "proposed_action": proposed_action})
    return {"proposed_action": proposed_action, "approved": bool(approved)}
```

`_build_proposed_action` is safe only because it is a pure function of checkpointed state. If anyone ever put an LLM call or a live GitHub read above `interrupt()`, the comment posted after approval could differ from the one the human approved — silently (DD §2.4 F2). The fix is to hash the payload, include the hash in the interrupt value, and require the resume to echo it (DD §1.9 does this with `patch_sha`).

**What the checkpointer does NOT give you** (say these unprompted — DD §4.4):

- **Mutual exclusion.** Two `POST /api/resume` requests can both load the paused checkpoint and both run `execute` → two comments (DD §2.4 F1). Fix: per-thread advisory lock or lease, and an idempotency key on the write.
- **Exactly-once side effects.** A node that posted to GitHub and crashed before its checkpoint will re-run (F3). Fix: marker in the comment body + read-before-write, or an `executed_actions` unique key.
- **Approval expiry.** A paused thread waits forever (F7). Fix: TTL + re-validate evidence at resume.
- **Schema migration of paused state.** A thread paused under v1 of `GraphState` resumes under v2. Fix: `total=False` + defaults, versioned state with upgraders, or drain before incompatible deploys.
- **Secrets hygiene.** ResolveFlow puts `github_token` *in state*, so it's in the checkpoint tables. Better: store a `credential_ref`, resolve it at execution time.

### 1.1.7 Subgraphs

A **subgraph** is a compiled graph used as a node of another graph. Two ways:

1. **Shared state keys:** `parent.add_node("review", review_subgraph)` — the subgraph reads and writes the parent's keys that it shares. Simplest.
2. **Different schema:** call the subgraph *inside* a node function, mapping parent state in and results out. Use when the subgraph has private working state that should not leak into the parent.

The parent's checkpointer propagates to the subgraph, so an `interrupt()` inside a subgraph pauses the whole run and resumes correctly; `get_state(config, subgraphs=True)` shows nested state. Why subgraphs matter for multi-agent design: each specialist can be its own graph with its own state schema and its own tests, composed by a supervisor graph. That is exactly how you would rebuild Agentforce's Supervisor/Specialists in LangGraph (1.1.9).

### 1.1.8 Streaming (one paragraph, but know the names)

`graph.stream(..., stream_mode=...)`: `"values"` (full state after each step), `"updates"` (only each node's delta), `"messages"` (LLM tokens as they generate), `"custom"` (anything a node emits), `"debug"`. For an MR-authoring agent you would stream `updates` to the UI so the plan appears before the slow CI step finishes (DD §5.6: "stream intermediate state so the wait is legible").

### 1.1.9 The mapping — ResolveFlow ↔ Agentforce v2 ↔ a GitLab agent

| Concept | ResolveFlow (LangGraph, public) | Agentforce for Setup v2 (Daisy + AgentGraph) | GitLab Duo Workflow Service |
|---|---|---|---|
| Runtime | `StateGraph(GraphState)` | AgentGraph DSL on Agentforce 3.0 (Daisy) | LangGraph, Python gRPC service (public runbook) |
| Entry / triage | `classify` (rule-based: failing CI → deterministic; sparse → AI; else human) | `ConversationManager` calls `TriageRequestIntent` → `INFORMATION_SEEKING` / `ACTION_SEEKING` | Flow selection by trigger (issue, pipeline, @mention) |
| Knowledge / retrieval | `generate_diagnosis` retrieval: Pinecone k=3 over ~2,750 closed-issue chunks | `Knowledge Node`: product docs, NLDs, live Metadata & API grounding | Repo context, issues/MRs, Knowledge Graph |
| Planning | `generate_diagnosis` → structured `Diagnosis` (temp 0, `with_structured_output`) | `Reasoner Node`: High-Velocity Path or Dynamic Synthesis; `GatherMoreDetails` → scaffold → `compilePlan` | Planner agent / Software Development Flow |
| Independent check | `independent_review` (code-computed `groundedness_ok`, `risk_ok`, `permission_ok`; LLM writes only `reasoning`) | Plan validation (Stage C), allowlists, schema validation | CI as oracle; policy |
| HITL | `await_approval` → `interrupt()` | Durable approval of the exact resolved plan | MR review (remote flows); per-tool approval (IDE/chat) |
| Execution | `execute` — only side-effecting node; refuses without `approved` | `Executor Node` — no LLM, 100% fidelity, resolves data dependencies between steps | Executor in CI runner / IDE |
| Persistence | `AsyncPostgresSaver`, `thread_id = uuid4()` | Platform runtime state (session persistence not shipped per HLD/ART — HONEST GAP) | Checkpoints saved to GitLab (Rails) |
| Supervisor return | Graph ends at `END` | **Control always returns to CM after a specialist** | Flow orchestrator |

**How you would build Agentforce's shape in LangGraph (say this if he asks "could you rebuild it?"):** a supervisor graph whose state holds the conversation and a `mode`; the CM node routes with a conditional edge on the triage result; Knowledge, Reasoner and Executor are **subgraphs** with their own schemas; each specialist ends with `Command(graph=Command.PARENT, goto="cm", update={...})` so control returns to the CM; `GatherMoreDetails` is an `interrupt()` that surfaces clarifying questions; `compilePlan` Stage A is a `Send` fan-out (one vector search per scaffold step, same superstep, results merged by a reducer keyed on step id), Stage B is one structured-output LLM call; the Executor subgraph has **no model in it** and is the only place holding write credentials. "Control returns to the supervisor" is the property that makes it a *supervisor* pattern rather than a one-shot dispatch.

### 1.1.10 Worked example — trace one ResolveFlow run through the runtime

Issue: a GitHub issue with a failing CI check on its linked PR.

| Superstep | Node(s) | State written | Checkpoint |
|---|---|---|---|
| 0 | (input) | `issue_url`, `github_token` | #0 |
| 1 | `fetch_evidence` | `raw_evidence` | #1 |
| 2 | `normalize_evidence` | `evidence: IssueEvidence` | #2 |

| Superstep | Node(s) | State written | Checkpoint |
|---|---|---|---|
| 3 | `classify` | `classification = "deterministic"` (failing CI wins — precedence test `failing_ci_wins_even_with_sparse_body`) | #3 |
| 4 | `await_approval` | builds `proposed_action`, calls `interrupt()` → run returns `__interrupt__` to FastAPI, which shows it in the UI | #4 (with pending interrupt) |
| — | *(hours pass; container may recycle; Postgres holds #4)* | | |
| 5 | resume: `Command(resume=True)` → `await_approval` **re-runs from line 1**, `interrupt()` returns `True` | `proposed_action`, `approved=True` | #5 |
| 6 | `execute` | posts comment (+label, partial-success handled) | #6 |

**Self-test (answer out loud):**

1. Which line in ResolveFlow runs twice, and why is that safe today?
2. Two browser tabs click "Approve" at once — what happens, step by step, and what's the fix?
3. Why did swapping `SqliteSaver` → `AsyncPostgresSaver` not change any graph code?
4. What would break if you added a parallel second diagnosis node without touching `GraphState`?
5. Where would you put the lease, given the checkpointer doesn't provide mutual exclusion?
6. Explain `async` vs `sync` durability in one sentence each, and which you'd choose for `execute`.

---

## 1.2 MCP — the Model Context Protocol, as a protocol

**Resume lines:** "Model Context Protocol (MCP)"; "tool/function calling, structured tool contracts"; Salesforce S2 "MCP, REST APIs... onboarded eight enterprise capabilities to Headless 360 across API exposure, tool discovery, and MCP execution." **Status:** protocol facts **READY** (verified against modelcontextprotocol.io, Oct 2026); *which* eight capabilities, whose token reaches the MCP server, and what you personally built in Headless 360 → **PENDING** (PROBE §1, OCQ A7).

### 1.2.1 The idea, slowly

Before MCP, every agent framework had its own way to describe a tool (OpenAI function schema, LangChain `Tool`, Anthropic `tools`), and every integration was written N×M times: N agent hosts × M systems. MCP standardizes the **wire protocol between an AI application and a capability provider**, the way the Language Server Protocol standardized editor ↔ language tooling (the spec explicitly cites LSP as inspiration). Write a GitLab MCP server once; Claude Code, Cursor, GitLab Duo and your own LangGraph agent can all discover and call its tools.

**MCP vs. function calling — the distinction he may test:**

- **Function calling** is a *model API feature*: you send JSON-schema tool definitions in the prompt; the model emits a structured call; *your code* executes it. It says nothing about where the tool lives, how it's discovered, or how it's authorized.
- **MCP** is a *client-server protocol* for **discovering and invoking** tools (and resources and prompts) that live in a separate process or service. Inside the host, MCP tool definitions are typically *converted into* the model's function-calling format. So they're complementary layers: MCP is how the host *gets* the tools; function calling is how the model *asks* for one.

### 1.2.2 Architecture — Host, Client, Server

- **Host** — the LLM application the user interacts with (Claude Desktop, an IDE, your agent service). Owns the model, the user's consent, and security policy.
- **Client** — a connector *inside* the host; **one client per server connection**. The host creates clients; each maintains its connection to one server.
- **Server** — a service exposing capabilities (GitLab, GitHub, Salesforce metadata, a filesystem). Can be local (subprocess) or remote (HTTP).

Why one-client-per-server matters for security: servers are isolated from each other — a server cannot see another server's data or the full conversation; the host mediates everything.

### 1.2.3 Primitives

**Server features:**

- **Tools** — functions the *model* can call. Definition: `name`, `title`, `description`, `inputSchema` (JSON Schema), optional `outputSchema`, and `annotations` (`readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`). **The spec says annotations must be treated as untrusted unless the server is trusted** — a hint is not a policy.
- **Resources** — data the *application/user* can read, addressed by URI (`file://`, `gitlab://project/42/issues/7`), with `resources/list`, `resources/read`, templates.
- **Prompts** — templated messages/workflows a *user* selects (e.g., "/summarize-mr").

The control split is the design: **tools are model-controlled, resources are application-controlled, prompts are user-controlled.**

**Client features (2025-11-25): Sampling** (server asks the host's model to generate), **Roots** (server asks which directories/URIs it may operate in), **Elicitation** (server asks the user for more input). **In the current 2026-07-28 revision, Roots, Sampling and Logging are deprecated** (still functional in the deprecation window), and elicitation moved to the Multi Round-Trip Request pattern (below).

### 1.2.4 Wire protocol and lifecycle — two revisions you should know

**Base:** JSON-RPC 2.0. A request has `jsonrpc`, `id`, `method`, `params`; a response has the same `id` and either `result` or `error {code, message, data}`; a **notification** has no `id` and gets no response. Standard error codes: `-32700` parse error, `-32600` invalid request, `-32601` method not found, `-32602` invalid params, `-32603` internal error.

**Revision 2025-11-25 and earlier (what most deployed servers, likely including your Salesforce work, speak) — stateful, handshake-based:**

```
client → initialize {protocolVersion, capabilities, clientInfo}
server → result {protocolVersion, capabilities: {tools:{listChanged:true}, ...}, serverInfo}
client → notifications/initialized
client → tools/list              server → {tools:[{name, description, inputSchema, ...}]}
client → tools/call {name, arguments}
server → {content:[{type:"text", text:"..."}], structuredContent:{...}, isError:false}
```

Version and capabilities are negotiated **once per connection**. Streamable HTTP carried an `Mcp-Session-Id` header.

**Revision 2026-07-28 (current as of Oct 2026) — stateless.** Verified changelog highlights:

- The `initialize` / `notifications/initialized` handshake is **removed**. Every request carries its protocol version and client capabilities in `_meta` (`io.modelcontextprotocol/protocolVersion`, `.../clientCapabilities`); mismatches return `UnsupportedProtocolVersionError`.
- New mandatory `server/discover` RPC advertises supported versions, capabilities and identity.
- **Protocol-level sessions and `Mcp-Session-Id` removed.** Servers needing cross-call state mint explicit handles passed as ordinary tool arguments.
- **Multi Round-Trip Requests (MRTR)** replace server-initiated requests: a server returns `resultType: "input_required"` with `inputRequests`; the client retries the original request with `inputResponses`.
- **Tasks** (long-running operations with polling and durable handles) moved to an official extension.
- List results carry `ttlMs`/`cacheScope` caching hints; servers SHOULD return `tools/list` in a deterministic order (to help client caching and **LLM prompt-cache hit rates** — ties to 2.4).
- **OpenTelemetry trace context propagation in `_meta`** (`traceparent`, `tracestate`, `baggage`) is now documented — directly relevant to Part 5 and to Karan.

**Why the stateless change matters, in his register:** a stateless protocol is horizontally scalable behind an ordinary load balancer — any replica can serve any request because nothing is pinned to a connection. That's the same argument as "workers stateless between supersteps" (DD §6.7). **Say:** "The 2026 revision made MCP stateless — no initialize handshake, no session id; version and capabilities ride on every request — which is what you want for a multi-tenant gateway. Our Headless 360 work predates that, so it used the handshake-based revision" (confirm the revision you used — **PENDING**).

**Transports: stdio** (host launches the server as a subprocess; newline-delimited JSON-RPC over stdin/stdout — local tools, no network auth needed) and **Streamable HTTP** (single endpoint; client POSTs JSON-RPC; server responds with JSON or an SSE stream). The old HTTP+SSE transport is deprecated.

### 1.2.5 Authorization (the IAM questions he will ask)

For remote HTTP servers, MCP authorization is **OAuth 2.1**-based: the MCP server is an OAuth **resource server**; it publishes **Protected Resource Metadata** (RFC 9728) pointing to its authorization server; clients use PKCE; tokens must be **audience-bound** to the specific MCP server via **resource indicators** (RFC 8707); **token passthrough is forbidden** — an MCP server must not forward the token it received to a downstream API (it must obtain its own, appropriately scoped credential). The 2026-07-28 revision adds: validate the `iss` parameter (RFC 9207); client credentials are bound to the issuing authorization server; Dynamic Client Registration is deprecated in favor of Client ID Metadata Documents.

**Why "no token passthrough" is the confused-deputy rule:** if the MCP server forwarded your token, the downstream API could not tell which service is calling, audit would attribute actions wrongly, and a token minted for server A could be replayed at B. Audience binding makes a stolen token useless elsewhere. This is RFC 8693 token exchange — the same principle as your Salesforce `orgJWT` (Part 6).

### 1.2.6 Worked example — your Headless 360 bullet, explained with the protocol

"Onboarded eight enterprise capabilities to Headless 360 across API exposure, tool discovery, and MCP execution." Mechanically, per capability (DD §2.2 S2):

1. **API exposure** — the capability's entities must be reachable through a public API (Data or Tooling API). If not, it cannot onboard cleanly (AS Story D: documented as a platform gap rather than a bespoke workaround).
2. **Tool contract** — an MCP tool definition: name, description written for retrieval (the description is what the action-selection funnel embeds!), `inputSchema` with the exact fields/types, ideally `outputSchema`; annotations reflecting read vs. destructive.
3. **Tool discovery** — the host lists tools (`tools/list`), and with hundreds of tools you don't show them all to the model: you embed descriptions and retrieve top candidates per step — the `compilePlan` Stage A funnel (SG-GL §2) applied to MCP tools.

4. **MCP execution** — `tools/call` under an identity: **whose?** Either delegated (the user's token, narrowed by scope and audience) or a service identity with the user's authority re-checked downstream. **Know which (PENDING — OCQ A7).**
5. **Skill + permission mapping + eval cases** per capability.

#### 1.2.7 How a GitLab agent exposes/consumes MCP

GitLab publicly ships both directions (DD §1.2): Duo Agent Platform as an **MCP client** (connect external MCP servers to agents/flows), and GitLab as an **MCP server** (external agents act on GitLab), with OAuth tokens limited to `ai_workflows` / `mcp` scopes. DD §8.8 has the "design an MCP gateway" answer: audience-bound tokens, no passthrough, per-tool approval modes (always allow / ask / deny), per-client allowlists, rate limits per client and per user, **pinned and hashed tool definitions** (rug-pull detection — a server changing a tool's description after approval), tool outputs treated as untrusted data (they can carry injection), attribution to human + client.

**MCP-specific threats (AS §2.6.2):** tool poisoning (malicious instructions hidden in a tool description), rug pull (definition changes after approval), cross-server shadowing (one server's tool description manipulates how the model uses another's), and output injection.

**Self-test:**

1. Function calling vs. MCP — one sentence each, and how they compose.
2. What are the three server primitives and who controls each?
3. What changed in the 2026-07-28 revision, and why does statelessness help a multi-tenant gateway?
4. Why is token passthrough forbidden? Name the RFC for audience binding.
5. An MCP tool's annotation says `readOnlyHint: true`. Can your policy gate rely on it? Why not?

---

### 1.3 Multi-agent orchestration patterns (condensed; full table DD §4.3)

| Pattern | Use when | Where you've used it | Risk |
|---|---|---|---|
| Single ReAct loop | Exploratory, read-mostly | AI Bug Triage skill (Claude Code) | Reason/act boundary is a prompt behavior |
| Plan-then-execute | Steps knowable up front | Agentforce Reasoner → Executor | Plan stale if world changes mid-run |
| Generate → verify → iterate | Executable oracle (CI) | DD §1.5 MR agent | Converges on weakening tests unless the gate checks |
| Supervisor / specialists | Separable cognitive loads | Agentforce CM + Knowledge/Reasoner/Executor | Every handoff is an untrusted channel |
| Map-reduce (`Send`) | Parallel independent work | `compilePlan` Stage A per step | Budget blowup, partial failure |
| Critic / verifier | Grounding / review | ResolveFlow `independent_review` | LLM-grading-LLM is soft unless code decides |
| Reader / writer split | Untrusted text meets write capability | Bug Triage is read-only by construction | Spec can still carry injected intent |
| Event-chained flows | Lifecycle automation | — (GitLab's direction) | Feedback loops, identity across hops |

**The rule:** "Pick the least autonomous pattern that does the job; make every transition between parts a typed, checkpointed, deterministic edge." **Why multi-agent at all:** narrower tool sets per agent (least privilege — a reviewer with no write tool *cannot* write, ART §6), smaller prompts (cheaper, more accurate), independently testable components. **Why not:** every handoff adds latency, cost, and an injection channel; one agent with good tools beats five chatty agents.

**Agent-to-agent identity (DD §7.1):** agent B triggered by agent A inherits A's *triggering human*, intersected again with B's service account — never A's full rights. Record the chain in `upstreamTrigger`.

---

### 1.4 HITL & durable pause/resume (condensed; full depth DD §4.4–4.5, SG-GL §3)

**Status: READY (ResolveFlow), HONEST GAP (shipped at Salesforce).**

- **Snapshot, not replay:** resume from the latest checkpoint; never rebuild by replaying the event log, because replaying tool-call events can re-fire real side effects (re-grant access, re-post). The event log is the audit trail, not the resume source — the same reason databases recover from checkpoint + redo log.
- **Approve artifacts, not intentions:** show the exact diff/plan; bind approval to its hash; expire approvals; re-validate preconditions at resume.
- **Risk tiers (DD §4.5):** T0 reads → allow; T1 sandbox edits → allow (remote) / ask (local); T2 comment/label/draft MR → allow, audited; T3 CI config, deps, auth paths, ready-for-review → ask a CODEOWNER; T4 merge/release/deploy/protected settings → human only.
- **GitLab's design (DD §7.4):** remote flows have *no* per-call approval — the sandbox makes tool calls non-effects and the **MR is the approval point**. Your position: "Approve the artifact, not the keystrokes — and put the sandbox boundary and egress allowlist under security review."
- **Approval-fatigue detector:** rubber-stamp rate (approved in < 10 s on a > 100-line diff).
- **Segregation of duties:** triggering human can't be the sole approver (GitLab attributes flow MRs to the triggering user so "author can't approve own MR" applies).
- **Slack agent HITL (DD §2.2 S3):** signing-secret HMAC verification, 3-second ack then async processing, dedupe on event id (Slack retries), Block Kit buttons showing the exact change, approver ≠ requester for privileged changes, re-check authorization at execution time, token lifetime across the approval wait. Specifics → **PENDING** (OCQ A7).

---

### 1.5 The permission gate (condensed; taught in SG-GL §1, ART §3)

**Status: READY.** Four independent sequential checks — one fuzzy "validate the request" check cannot catch both failure modes below:

```
model proposes action
  1. GROUNDING  — is it real? entities exist, coherent against LIVE state    → catches hallucination
  2. PERMISSION — is THIS caller allowed THIS operation on THIS resource?    → catches unauthorized-but-
real
  3. APPROVAL   — high-impact? pause for a human                              → consequence checkpoint
  4. CONTRACT   — shaped exactly per the tool schema?                          → malformed calls
  execute
```

- Real user, caller lacks right to grant admin → grounding passes, **permission** fails.
- Hallucinated user "Jsmith", authorized caller → permission passes, **grounding** fails.
- **Evidence:** ~35% fewer misconfigured access-management requests from schema validation + operation allowlists (permission + contract stages working). Metric definition in DD §2.2 S5: relative reduction; know the absolute rates (**PENDING**).
- **The trap:** "the agent validates the request before executing" — sounds junior. Split them.
- **Where it lives in each system:** Agentforce — at the Executor (allowlist + independent authz
- human approval of the exact plan; AS Story E). ResolveFlow — `independent_review` (grounding via citation membership + relevance floor), `permission_ok` (honest scaffolding: only two low- risk writes exist), `interrupt()` (approval), `execute()` re-check + typed GitHub client (contract). GitLab — composite identity (permission), CI + policy gate on the artifact, MR approval rules.

**Hard vs. soft (AS §2.5.5, DD §4.6):** "Can the model talk its way past this?" A system-prompt rule is soft; a path-based policy on the diff, a sandbox with no push credential, an egress allowlist and composite identity are hard. "An injection classifier is a smoke detector, not a firewall."

---

# Part 2 — LLM, RAG & retrieval engineering

Resume lines covered here: "Retrieval Augmented Generation (RAG), adaptive & corrective RAG, semantic search, vector retrieval strategies, reranking, chunking & retrieval optimization, structured output extraction, prompt engineering, model selection & token economics, OpenAI / Anthropic / open-weight APIs, Pinecone, ChromaDB, FAISS, embeddings"; ResolveFlow's "~2,750 closed-issue chunks with measured relevance thresholds"; AdaptiveRAG's "naive, reranked, corrective, and adaptive retrieval with query-classification routing over a 100-question evaluation set — +12% groundedness, –21% token cost, –18% median latency."

**Status overview:** concepts **READY** (papers verified). ResolveFlow retrieval numbers **READY** (from code + CHANGELOG). AdaptiveRAG implementation specifics (which reranker, which evaluator, how groundedness was scored, relative vs. percentage points) → **PENDING**: the code is not in this repo; open it before Oct 9 and fill in 2.1.8.

## 2.1 RAG variants, precisely

### 2.1.1 The idea, slowly

RAG = before the model answers, **retrieve** relevant text from an external corpus and put it in the prompt, so the answer is grounded in evidence rather than the model's parametric memory. Every RAG variant is an answer to one question: **what should happen when retrieval is unnecessary, insufficient, or wrong?**

| Variant | Its answer | Key mechanism |
|---|---|---|
| Naive RAG | Always retrieve top-k, always use it | Embed query → ANN search → stuff k chunks → generate |
| Reranked RAG | Retrieve wide, then re-order precisely | Bi-encoder top-N (e.g., 50) → cross-encoder rerank → top-k (e.g., 5) |
| Corrective RAG (CRAG) | **Grade** what was retrieved; correct it if bad | Retrieval evaluator → Correct / Incorrect / Ambiguous → refine or web-search |
| Self-RAG | The *generator itself* decides when to retrieve and critiques its use | A model trained to emit reflection tokens |
| Adaptive RAG | **Classify the query first**; choose how much retrieval it needs | Complexity classifier → no-retrieval / single-step / multi-step |

The one-sentence distinction to say: "*Corrective RAG fixes bad retrieval after it happens; adaptive RAG avoids unnecessary or insufficient retrieval before it happens; Self-RAG folds both decisions into the generator through trained reflection tokens.*"

### 2.1.2 Naive RAG — the baseline and its failure modes

```
query → embed → top-k by cosine → prompt = instructions + k chunks + query → LLM
```

Failure modes: (1) **retrieval miss** — the right chunk isn't in top-k (recall problem); (2) **distractors** — irrelevant chunks dilute attention and can carry injection; (3) **forced grounding** — the model cites whatever came back even when nothing is relevant (exactly what your ResolveFlow relevance floor fixed — Story E); (4) **wasted retrieval** — "what is 2+2?" still pays for an embedding call, a vector search, and ~k×chunk tokens; (5) **one-shot** — a multi-hop question ("which maintainer fixed the bug that broke X in the version after Y?") needs iterative retrieval.

### 2.1.3 Reranked RAG

Stage 1, a **bi-encoder** (embeddings) retrieves wide for **recall** (top 50). Stage 2, a **cross-encoder** reads (query, chunk) *together* and scores relevance for **precision**; keep top 5. Bi-encoders can precompute document vectors

(fast); cross-encoders cannot (slow, so only on a shortlist). Full teaching in 2.3.1. **Cost/latency effect:** adds a reranker call (tens to hundreds of ms) but lets you send **fewer, better** chunks to the LLM — often a net token saving.

### 2.1.4 Corrective RAG (CRAG) — the actual technique

Source: Yan, Gu, Zhu, Ling, *Corrective Retrieval Augmented Generation*, arXiv 2401.15884 (2024). Verified details:

1. **Retrieval evaluator** — a lightweight model (**fine-tuned T5-large, ~0.77B params**) scores each (question, document) pair for relevance (roughly −1 to 1).
2. **Confidence → one of three actions**, using an upper and lower threshold (dataset-specific; e.g., 0.59 / −0.99 for PopQA): - **Correct** — at least one document scores above the upper threshold → use retrieved docs, after refinement. - **Incorrect** — all documents below the lower threshold → discard them; **web search** instead. - **Ambiguous** — in between → combine refined internal knowledge **and** web results.
3. **Knowledge refinement — "decompose-then-recompose":** split each retrieved document into **knowledge strips** (a few sentences each), score each strip with the evaluator, drop irrelevant strips (paper uses a −0.5 cut), recombine what survives. This removes distractor text inside otherwise-relevant documents.
4. **Web-search query rewriting:** the question is rewritten into **keyword queries** (they used ChatGPT) because search engines work better on keywords than on full questions.
5. It's **plug-and-play**: wraps any RAG pipeline.

**The engineering pattern, generalized (what you'd actually build):** a *grader* node after retrieval → if grade is low: rewrite the query and re-retrieve, widen to another source, or abstain/escalate; if mixed: filter at strip level. In LangGraph: `retrieve → grade → (generate | rewrite → retrieve | web_search)` with conditional edges — the canonical LangGraph CRAG tutorial shape.

**Tie to ResolveFlow (be precise — it's *partially* corrective):** `independent_review` computes `groundedness_ok` from citation membership **and** a relevance floor (`retrieved score ≥ 0.35`); a failing grade **escalates to a human** rather than re-retrieving. The `reject_retrieve_more` route exists in `build.py` but **is not built** (it routes to `END`; docstring says so). So: "ResolveFlow implements CRAG's *evaluator* and its *Incorrect → don't use it* action; the corrective re-retrieval loop is wired but intentionally not built — escalation to a human is the correction." That's honest and precise.

### 2.1.5 Self-RAG — know it to contrast it

Source: Asai, Wu, Wang, Sil, Hajishirzi, *Self-RAG*, arXiv 2310.11511 (2023). The LM is **trained** to emit **reflection tokens**: `Retrieve` (should I retrieve now?), `ISREL` (is this passage relevant?), `ISSUP` (is my statement supported by it?), `ISUSE` (is the response useful?). Retrieval becomes on-demand and segment-level; at inference you can weight the critique tokens to trade precision vs. fluency. **Difference from CRAG:** Self-RAG requires a specially trained generator; CRAG uses a separate small evaluator and works with any generator. **Why you likely didn't use Self-RAG in AdaptiveRAG:** you used Claude via API — you can't add trained reflection tokens to a hosted model; you can only *emulate* them with prompted grader calls.

### 2.1.6 Adaptive RAG — the actual technique

Source: Jeong, Baek, Cho, Hwang, Park, *Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity*, NAACL 2024 (arXiv 2403.14403). Verified:

- A **smaller LM classifier** predicts query complexity; labels are **collected automatically** (from which strategy actually answered training questions correctly, plus dataset bias — single-hop datasets lean simple, multi-hop lean complex), so no manual annotation.
- Three routes: **(A) no retrieval** — the LLM answers from parametric knowledge; **(B) single-step retrieval**; **(C) iterative multi-step retrieval** (interleaving retrieval and reasoning, IRCoT-style).

- Result: better accuracy *and* efficiency than always-single-step (wastes effort on simple questions, fails on complex ones) or always-multi-step (accurate but slow and expensive).

**"Query-classification routing" in your resume = this idea.** In practice (and likely in your platform — **confirm**), the classifier can be a prompted small LLM or a rules+embedding classifier rather than a fine-tuned T5, and the routes can be strategies rather than step counts: e.g., *no-retrieval*, *naive single-shot*, *reranked*, *corrective/iterative*.

### 2.1.7 Comparison table to draw on a whiteboard

| | **Naive** | **Reranked** | **Corrective** | **Adaptive** |
|---|---|---|---|---|
| Decides **whether** to retrieve | No | No | No | **Yes** (class A skips) |
| Decides **how much** | No | No | Reactively (re-retrieve) | **Yes**, before retrieval |
| Checks **quality** of retrieved docs | No | Implicitly (rerank score) | **Yes — evaluator** | Depends on route |
| Extra model calls | 0 | +1 reranker | +evaluator (+rewrite +search) | +1 classifier (cheap) |
| Typical cost vs naive | baseline | ≈ or lower LLM tokens (fewer chunks) | higher on hard queries | **lower on average** |
| Typical latency | baseline | +rerank ms | higher (loops) | lower median, higher tail |
| Fails when | Retrieval misses / noise | Recall miss in stage 1 | Evaluator miscalibrated | Classifier misroutes a hard query as easy |

### 2.1.8 Your AdaptiveRAG numbers — how to defend them

"+12% groundedness, –21% token cost, –18% median latency vs. a fixed baseline, 100 questions."

**Why adaptive is cheaper and faster (mechanism — READY):** the router sends easy/factual queries down **no-retrieval** or **single-shot** paths (no embedding call, no k chunks in the prompt, no grading loop) and reserves corrective/iterative retrieval for the queries that need it. Token cost falls because the average prompt is shorter; **median** latency falls because the median query is easy. Expect **p95 latency to rise or stay flat** — hard queries now take the iterative path. Say that unprompted: "*median improves; the tail is where the expensive route lives.*"

**Why groundedness rises:** corrective grading removes irrelevant chunks (fewer distractors, less forced citation), and hard queries get multi-step retrieval instead of a single shot that misses.

**The five questions to pre-answer (DD §2.1 template) — PENDING until you open the code:**

1. **Unit:** is +12% *relative* (e.g., 0.70 → 0.784) or *percentage points* (70% → 82%)? Say which.
2. **Scoring:** how was groundedness measured? LLM-as-judge with what rubric (claim-level support against retrieved context?), which judge model, was it calibrated against your own labels (agreement / Cohen's κ)?
3. **Baseline:** "fixed baseline" = always naive top-k? same k? same model?
4. **Statistics:** n = 100. If groundedness is per-question binary, a 95% interval on a proportion near 0.7 is about ±9 points — so a 12-point gap is near the edge unless you used a **paired** test on the same 100 questions (McNemar on discordant pairs), which is much more powerful. If it's a continuous score, use a paired bootstrap CI or Wilcoxon signed-rank. Saying this unprompted is a Fellow-grade move (DD §4.7.4).
5. **What moves it:** classifier accuracy (what % misrouted?), question mix (more multi-hop → less saving), judge model change.

**Infra side (Part 4.1):** Lambda + API Gateway + FAISS — how did FAISS load (bundled in the deployment package / container image, or pulled from S3 at init)? Cold-start time with the index? Memory size? → **PENDING**, and it's your most hands-on AWS example, so it matters most.

**Self-test:**

1. One sentence each: naive, reranked, corrective, adaptive, Self-RAG.
2. What are CRAG's three actions, and what does "decompose-then-recompose" remove?
3. Why does adaptive RAG lower *median* latency but possibly raise *p95*?
4. ResolveFlow: which part of CRAG does it implement, and which part is deliberately not built?
5. Is "+12% groundedness" on 100 questions significant? What test, and why paired?

---

## 2.2 Vector databases — Pinecone vs. ChromaDB vs. FAISS

**Status: READY** for concepts and ResolveFlow (Pinecone, verified in `ingest.py` / `tools/retrieval.py`); AdaptiveRAG's FAISS index type and ChromaDB usage context → **PENDING** (where did you use Chroma? Be ready to say).

### 2.2.1 The idea — what each one actually *is*

These are three different kinds of thing, and saying so is the first signal of depth:

- **FAISS** (Facebook AI Similarity Search, Meta) — a **C++ library with Python bindings** for similarity search over dense vectors. Not a database: no server, no auth, no built-in metadata filtering beyond ID selectors, no replication; persistence is `faiss.write_index` / `read_index` to a file. You embed it in your process. Extremely fast, GPU support, the widest menu of index types.
- **ChromaDB** — an **open-source vector database** designed to be developer-first. Runs **embedded** (in-process; `chromadb.PersistentClient(path=...)` stores to local disk) or **client-server** (HTTP), plus a hosted cloud. Has collections, documents + metadata, metadata `where` filters, pluggable embedding functions. Uses **HNSW** (hnswlib) per collection; the core was rewritten in Rust (~2025).
- **Pinecone** — a **fully managed, cloud-hosted vector database** (SaaS). You never run it. Serverless indexes (storage/compute separated, pay per read/write units + storage), **namespaces** for partitioning, metadata filtering at query time, sparse-dense (hybrid) support, managed scaling and replication; also integrated inference/reranking offerings.

### 2.2.2 Index algorithms you must be able to explain

| Index | How it works | Knobs | Trade-off |
|---|---|---|---|
| **Flat** (exact) | Compare query to every vector | — | Perfect recall, O(N) per query; fine to ~100k–1M vectors on CPU |
| **IVF** (inverted file) | k-means into `nlist` clusters at train time; at query, search only the `nprobe` nearest clusters | `nlist`, `nprobe` | Needs a training step; recall depends on `nprobe`; good for large, mostly-static corpora |
| **PQ** (product quantizatio n) | Split each vector into m sub-vectors, quantize each to a codebook id → huge compression | m, bits | Big memory saving, lossy distances; often IVF-PQ |
| **HNSW** | Layered proximity graph: greedy search from sparse top layers ("highways") down to the dense bottom layer | `M` (links per node, build time), `efConstruction`, `efSearch` (query time) | High recall + low latency, memory-heavy, supports incremental inserts; deletes are awkward |

From RAG-S §3: `M` is baked in at build (changing it means rebuild); `ef_search` is your per-query recall/latency dial.

**Distance metrics:** cosine (direction only), dot product (= cosine if vectors are L2-normalized), L2 (Euclidean). OpenAI embeddings are normalized, so cosine ≈ dot. ResolveFlow's Pinecone index metric is **cosine**.

### 2.2.3 Trade-offs

| Concern | FAISS | ChromaDB | Pinecone |
|---|---|---|---|
| Hosting | In your process | In-process or self-hosted server or Chroma Cloud | Fully managed SaaS |
| Ops burden | You build everything around it | Low | Lowest |
| Persistence | Manual file save/load | Built-in (local disk / server) | Built-in, replicated |
| Metadata filtering | Minimal (ID selectors; you build the rest) | Yes (`where`) | Yes, at query time |
| Updates / deletes | Possible, awkward for some indexes | Yes | Yes (upsert/delete by id) |
| Multi-tenancy | You build it | Collections / tenants | **Namespaces** + metadata filters |
| Scale | Billions with IVF-PQ + GPU, single-node unless you shard | Single-node strong; distributed via cloud | Managed horizontal scale |
| Cost | Free + your compute | Free (OSS) / cloud pricing | Usage-based |
| Data residency / air-gap | Excellent (all local) | Excellent (self-host) | Depends on region offering |

### 2.2.4 Which you'd pick, and why (the answer he wants)

- **FAISS** — when the index is **read-mostly and fits on one box**, latency must be minimal, and you can rebuild offline: e.g., **AdaptiveRAG** — a fixed eval corpus, loaded into a Lambda container at init; no server to run, no network hop. Caveat you must volunteer: on Lambda the index is **reloaded on every cold start**, and each concurrent container holds its own copy in memory.
- **ChromaDB** — prototypes, local dev, small-to-medium self-hosted apps, or **air-gapped / self-managed** deployments where data must not leave the customer's infra.
- **Pinecone** — production multi-user service with continuous upserts, metadata filtering and no desire to operate a database: **ResolveFlow** — serverless index, weekly re-ingest, zero ops on a free-tier deployment.
- **pgvector** (not on your resume, but say it) — if you already run Postgres, a vector column with HNSW keeps embeddings next to the rows they describe, inside the same transaction and the same ACLs. For GitLab specifically — a Postgres shop with self-managed customers — this is likely the boring, correct default; mention it as a reasoning point, not a claim about GitLab's implementation.

### 2.2.5 Multi-tenancy in a vector store (his specialty)

Options: one index per tenant (strong isolation, operational overhead), **namespace/partition per tenant** (Pinecone namespaces — the query can't cross namespaces), or a shared index with a mandatory `tenant_id` **pre-filter** inside the query. Never post-filter (RAG-S §7: post-filtering leaks via the shortfall and exposes restricted text to anything that reads the candidate set). Salesforce's version is structural: the retriever only ever receives a tenant-scoped `orgJWT` (Part 6). Cache keys for retrieval results must include tenant + authorization context.

### 2.2.6 ResolveFlow's retrieval, exact (READY)

- Corpus: 4 repos (`facebook/react`, `langchain-ai/langchain`, `microsoft/terminal`, `vercel/next.js`) × 150 closed issues = 600 issues → **~2,750 chunks** (~4.6 per issue).
- Splitter: `RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=40)`.
- Embeddings: `text-embedding-3-small` at **512 dims** (Matryoshka truncation — RAG-S §2).
- Store: Pinecone serverless index `resolveflow-issues`, metric cosine; `ingest.py` does delete-all then upsert (idempotent re-ingest); weekly `reingest.yml`.
- Query: `similarity_search_with_score(query, k=3)` where query = title + body.

- Gate: `MIN_RELEVANCE_SCORE = 0.35` between the measured noise cluster (0.21–0.22; later 0.24–0.26) and signal cluster (0.53–0.63; later 0.55–0.66). **R11 arithmetic trap:** midpoint is 0.375 (or 0.405) — 0.35 leans slightly toward *approving*, not escalating; don't claim the opposite (DD §3.5).
- Coupling: the threshold is coupled to `EMBEDDING_MODEL`/`EMBEDDING_DIMENSION`; change the model → recalibrate. Production version: pick τ from a precision–recall curve on ~200 labeled pairs.

**Note correcting RAG-S §2:** RAG-S says related text "scores 0.7–0.999." That's model-dependent. For `text-embedding-3-small` your own measurements show *relevant* matches at ~0.55–0.66. The point that stands: **absolute cosine values are model-specific; calibrate per model; rank is more reliable than raw score.**

**Self-test:**

1. Why is FAISS "not a database"? What would you have to build around it for production?
2. IVF vs HNSW — which needs a training step, which supports incremental inserts well?
3. Pick a store for: (a) Lambda-hosted eval over a fixed corpus; (b) multi-tenant SaaS with continuous updates; (c) a self-managed customer who forbids egress. Justify each.
4. How do Pinecone namespaces help tenant isolation, and why is a post-filter not enough?

---

## 2.3 Reranking, chunking, embeddings

**Status: READY** (concepts); RAG-S has the plain-language versions — this section adds depth.

### 2.3.1 Embeddings — what they actually represent

An embedding model maps text to a fixed-length vector such that **semantic similarity ≈ geometric closeness**. Modern text embedders are transformer encoders trained with **contrastive learning**: pull (query, relevant passage) pairs together, push in-batch non-matches apart. Individual dimensions mean nothing; only relative geometry does. Consequences:

- **Query and document must use the same model and dimension** (ResolveFlow's retrieval.py docstring says exactly this) — different models are different spaces.
- **Asymmetric retrieval:** queries are short, passages long; some models use instruction prefixes ("query:" / "passage:") to handle this.
- **Matryoshka representation learning:** models like `text-embedding-3-*` are trained so the first N dimensions carry most information; you can truncate (1536 → 512) for ~3× less storage and faster search at a small recall cost — which is why ResolveFlow uses 512.
- **What embeddings are bad at:** exact identifiers (SKUs, error codes, commit SHAs, function names), negation ("did NOT close"), numbers. → hybrid search with BM25 (RAG-S §4) and cross-encoder reranking. **For code agents specifically:** symbols, call graphs and references are exact — use a code graph or exact search, not embeddings (DD §4.12).

### 2.3.2 Reranking — bi-encoder vs. cross-encoder vs. late interaction

| | **Bi-encoder** | **Cross-encoder** | **Late interaction (ColBERT)** |
|---|---|---|---|
| Input | Query and doc encoded **separately** | Query **and** doc concatenated, one pass | Separately, but keeps **per-token** vectors |
| Precompute docs? | Yes | **No** | Yes (token vectors) |
| Scoring | One dot product | Full attention across both texts → relevance score | Sum over query tokens of max similarity to doc tokens (MaxSim) |
| Speed | Fastest — ANN over millions | Slow — only over a shortlist (tens–hundreds) | Middle; more storage |

| | Bi-encoder | Cross-encoder | Late interaction (ColBERT) |
|---|---|---|---|
| Quality | Good recall | Best precision | Near cross-encoder, faster |

**Why a cross-encoder is more accurate:** attention can relate every query token to every document token — it sees that "did the deal close?" vs. "the deal did NOT close" is a mismatch, which a single averaged vector blurs. **Why it can't be the first stage:** cost is O(N) model forward passes per query.

**Two-stage pipeline:** `ANN top-50 (recall) → cross-encoder → top-5 (precision) → LLM`. Examples: open cross-encoders (ms-marco MiniLM family, BGE rerankers), hosted rerank APIs (Cohere, Pinecone, Voyage). An **LLM-as-reranker** (listwise) is a third option: most flexible, most expensive. **When to skip reranking:** the candidate set is already small and clean (ACL-narrowed to a few dozen docs), or latency budget is tight and recall@5 already meets target.

**Measure it:** recall@k for stage 1 ("is the right chunk in the top 50 at all?"), nDCG@5 / MRR for stage 2 ("is it at the top?"). If stage-1 recall is the problem, reranking can't help.

### 2.3.3 Chunking strategies and trade-offs

| Strategy | Mechanism | Pros | Cons |
|---|---|---|---|
| Fixed-size | Every N tokens/chars, overlap O | Simple, predictable | Cuts sentences, tables, code blocks |
| **Recursive** (ResolveFlow) | Try split on `\n\n`, then `\n`, then space, then char, until ≤ N | Respects structure where possible; good default | Still size-driven, not meaning-driven |
| Structural / document-aware | Split on Markdown headers, HTML sections, code functions/classes (AST) | Chunks align to units of meaning; can carry header path as metadata | Needs per-format parsers |
| Semantic | Embed sentences; start a new chunk where similarity between neighbors drops (percentile breakpoint) | Topic-coherent chunks | Extra embedding cost at ingest; variable size |
| Parent-child / small-to-big | Index small chunks for precise matching, return the larger parent for context | Precision *and* context | More storage, more complex |
| Contextual retrieval | Prepend an LLM-written 1–2 sentence context ("This chunk is from the 2024 React issue about…") to each chunk before embedding and BM25 | Anthropic reported ~49% fewer failed retrievals (contextual embeddings + contextual BM25), ~67% with reranking (Sept 2024) | LLM cost per chunk at ingest (prompt caching makes it affordable) |

**The size trade-off (RAG-S §1):** too small → chunk lacks context; too big → its embedding averages several topics and matches none strongly ("the single most common silent RAG failure"). **Overlap** gives boundary-straddling facts a second chance. **800/40 is a starting point; measure recall@k on a real query set and tune.**

**ResolveFlow-specific critique to volunteer:** an issue is title + body; recursive 800-char chunks can separate the title (the most informative line) from later chunks of the same issue. Improvement: prefix every chunk with the issue title and repo (cheap contextual retrieval), and store `issue_number` / `repo` / `labels` as metadata to enable filtered search.

### 2.3.4 Structured output extraction (resume skill)

`llm.with_structured_output(Diagnosis)` in ResolveFlow binds a Pydantic schema: depending on provider it uses native JSON-schema structured outputs or tool calling under the hood, and returns a validated object (or raises). Why it matters: downstream code (`independent_review`) reads typed fields (`citations`, `severity`) — no regex over prose. **Limits to volunteer:** schema-valid ≠ correct (a valid `severity: "low"` can be wrong); constrained decoding guarantees shape, not truth → that's why the gate is a separate deterministic check. Validate enums in code (`risk_ok = severity in {"low","medium"}`) rather than trusting the model's enum discipline.

**Self-test:**

1. Explain to a non-ML engineer why a cross-encoder is more accurate but can't search millions of docs.
2. Why must query and document embeddings come from the same model and dimension?
3. When would semantic chunking beat recursive? When is it not worth the cost?
4. What's one concrete chunking improvement for ResolveFlow's corpus?

---

## 2.4 Model routing & token economics

**Resume lines:** "model routing," "model selection & token economics," "OpenAI / Anthropic / open-weight APIs," "latency & cost analysis"; ResolveFlow "benchmarking OpenAI, Anthropic, and open-weight models on routing accuracy, behavior, and cost — 86% routing accuracy at 24% lower execution cost than baseline." **Status:** concepts **READY**; **the 86% / 24% benchmark is the highest-risk claim on the resume (DD §0.3 R8): not in the public repo, only `gpt-4o-mini` is called, `classify()` is rule-based.** Resolve before Oct 9 per DD §4.7.6 (commit it, build it, or say it out loud first).

### 2.4.1 Two different meanings of "routing" — don't blur them

1. **Workflow routing** — which *path* a request takes (ResolveFlow `classify`: deterministic / ai_investigation / human_review; Agentforce triage; Adaptive-RAG's query classes). "Routing accuracy" in R8 = % of scenarios whose final route matches the labeled expected route.
2. **Model routing** — which *model* serves a given call (small vs. large, provider A vs. B, hosted vs. open-weight).

### 2.4.2 Why route between models

Models differ on four axes: **capability** (reasoning depth, tool-use reliability, long context), **cost** (per-token price, often 10–30× between small and frontier tiers), **latency** (time to first token and tokens/sec), **constraints** (data residency, self-hosting, rate limits). No single model is best on all four, and **most calls in an agent don't need the best model**: classifying an issue, extracting fields, grading a chunk's relevance are small-model tasks; synthesizing a multi-file patch is not.

**Routing strategies, in increasing sophistication:**

- **Static per-node assignment** — choose a model per graph node based on eval results (cheapest model that holds the node's quality bar). This is the right first step and what DD §4.7.6 proposes for ResolveFlow.
- **Rule-based** — by input features (length, language, presence of code, tenant tier).
- **Classifier-based** — a cheap model predicts difficulty and picks the tier (same idea as Adaptive-RAG, applied to models).
- **Cascade / fallback** — try the small model; if a verifier (schema check, confidence, self-consistency) fails, escalate to the large model. Pays double on escalations; wins when most calls succeed small.
- **Availability fallback** — on provider degradation (DD §9.7) switch to a secondary model **that has passed the eval suite for that flow** — an unevaluated fallback is a silent behavior change.

**Rule (DD §4.12):** "Big model everywhere early, before you have evals; per-step routing once evals can prove the cheap model holds on that step." Routing without evals is guessing.

### 2.4.3 Token economics — what it concretely means

**Cost of one call** = input_tokens × input_price + output_tokens × output_price (output is typically ~4–5× input price per token). **Cost of an agent run** = Σ over calls, and in a loop the *input grows every turn* because the full transcript is resent (ART §1: the transcript *is* the memory). An 8-turn loop with a 6k-token system+tools prefix and ~1k new tokens per turn sends roughly 6k+7k+8k+…+13k ≈ 76k input tokens, not 8×7k = 56k — **context growth is superlinear cost**. That's the economic argument for compaction (SG-GL §4) and for plan-then-execute over long ReAct loops.

**Illustrative arithmetic** (prices are placeholders — say "illustrative" and check the current price sheet): a call with 6,000 input and 800 output tokens at $3/M input and $15/M output = 0.018 + 0.012 = **$0.03**. The same call on a small model at $0.15/M in, $0.60/M out ≈ **$0.0014** — ~20× cheaper. If 70% of an agent's calls are classification/extraction that a small model handles at the same eval score, routing them cuts run cost by well over half.

**The cost drivers, ranked (what you optimize, in order):**

1. **Number of calls** (loop iterations, retries, corrective loops) — eliminate calls first (deterministic triage on the High-Velocity Path; Adaptive-RAG's no-retrieval route).
2. **Input size per call** — retrieved chunks (k × chunk size), tool definitions (the funnel's whole point is showing 3–5 candidates instead of hundreds), transcript growth.
3. **Model tier per call** — routing.
4. **Output length** — `max_tokens`, structured output (no prose preambles).
5. **Caching** — below.

**Prompt caching (verified, Anthropic docs, Oct 2026):** the provider caches the processed prefix of a prompt; requests sharing that exact prefix pay less and start faster. Anthropic: `cache_control` breakpoints (up to 4), prefix order **tools** → **system** → **messages** (changing tool definitions invalidates everything after), 5-minute TTL default (cache writes at 1.25× base input price) or 1-hour TTL (2×), **cache reads at ~0.1× base input** (lower on some newest models), minimum cacheable length model-dependent (512–4,096 tokens). OpenAI: automatic caching for prompts ≥ 1,024 tokens with a model-dependent discount (check current docs). **Design rules that follow:** put stable content first (tool schemas, system prompt, few-shot examples, Skill/NLD text), volatile content last (user query, retrieved chunks, timestamps); keep tool order deterministic (the MCP 2026 spec asks servers to return `tools/list` deterministically for exactly this reason); never put a timestamp in the system prompt.

**Other levers:** batch APIs (~50% discount for non-interactive work — eval suites, nightly re-ingest, contextual-retrieval chunk annotation); semantic caching of whole answers (dangerous: key must include tenant, authz context and data version — AS Story C cache-key bug); smaller embedding dims (storage, not LLM tokens).

**Unit economics he'll ask for (DD §4.7.2):** cost per **successful** task (cost per merged MR, not per run), because a cheaper model that fails more costs more per success. **"An agent loop's cost envelope is a distribution"** (POST) — report p50 and p95 cost per run, and alert on outliers (DD §9.5: one tenant's cost tripled → runs hitting the iteration cap).

**Latency economics:** time-to-first-token is dominated by input size (prefill) and caching; total time by output length and number of sequential calls. Parallelize independent calls (`Send`; HLD's proposed `execution_mode: "parallel"` stage for independent lookups); stream.

#### 2.4.4 OpenAI / Anthropic / open-weight — what actually differs for an engineer

- **API shape:** both support tool/function calling with JSON schemas, structured outputs, streaming, prompt caching (explicit breakpoints on Anthropic, automatic on OpenAI). Message formats differ (system prompt as a top-level field on Anthropic). A thin provider abstraction (the "model gateway" — POST §9) keeps nodes provider-agnostic; LangChain's `ChatOpenAI` / `ChatAnthropic` is one such abstraction.
- **Open-weight models** (Llama, Mistral, Qwen, gpt-oss etc.) served via vLLM / TGI / Ollama, often behind an **OpenAI-compatible endpoint** — so the same client code works. Why choose them: data residency and **self-managed / air-gapped customers (GitLab has these)**, cost at high steady volume, fine-tuning. Costs: you own GPU capacity, batching, quantization trade-offs, and usually weaker tool-calling reliability → stricter schema validation and retries.
- **Honest ResolveFlow status:** the CHANGELOG records switching the provider from Anthropic to OpenAI; only `gpt-4o-mini` is called today. Don't imply a live multi-provider router exists in the repo unless you've built it.

#### 2.4.5 Worked example — designing ResolveFlow's model routing (what you'd say)

"Two LLM nodes: `generate_diagnosis` and the `independent_review` critique. The critique only writes `reasoning` — the decision is code — so it's the safest place for the cheapest model. Diagnosis quality drives everything downstream, so I'd hold it on the stronger model until the benchmark shows the cheap one keeps groundedness and routing accuracy within a pre-declared margin, using a paired McNemar test on the same 50 scenarios, reporting cost per *correct* route and p50/p95 latency. Classification is already rule-based, so it costs zero tokens — the cheapest routing decision is the one you don't send to a model."

**Self-test:**

1. Define "routing accuracy" in ResolveFlow precisely. What would the confusion matrix rows be?
2. Why does an 8-turn loop cost more than 8 × one turn?
3. Order tools, system prompt, retrieved chunks and the user question for maximum cache hits.
4. When is a cascade (small → large on failure) worse than always-large?
5. What do you say if he asks to see the 86% benchmark and it isn't in the repo?

---

## Part 3 — Evaluation, testing, safety & guardrails

Resume lines: "AI evaluation frameworks, regression / retrieval / routing evaluation, latency & cost analysis, groundedness & hallucination reduction, model performance monitoring, guardrails, prompt-injection defense, input validation, output sanitization, permission-aware execution, data governance"; Salesforce S4/S5; ResolveFlow's 50-scenario suite. Full depth: DD §4.6–4.7, §2.2 S4, AS §2.5. This part condenses and adds what's missing (output sanitization, input validation layers, data governance, hallucination taxonomy).

### 3.1 Evaluation framework design (condensed from DD §4.7)

**Layers (say them as a list):**

1. **Component / unit:** retrieval recall@k, MRR, nDCG against labeled (query → relevant doc) pairs; tool-selection exact match + argument-schema validity; structured-output schema-valid rate; routing accuracy vs. labeled route.
2. **Trajectory:** iterations, tools used, dead ends, budget consumed — catches "right answer, absurd cost."
3. **End-to-end task:** for code agents, SWE-bench-style — repo at pre-fix SHA, hidden tests from the human fix; resolved = fail-to-pass now pass and pass-to-pass still pass.
4. **Safety / regression gates — pass^k at 100%:** adversarial cases with binary expected outcome (refused/escalated). One failure in k trials blocks release. ResolveFlow `eval/harness.py` ("one bypassed

gate across any number of trials is a critical failure, never something to average away").
5. **Online:** same scorers on sampled production traces + outcome labels (merged, reverted, human edit distance).

**pass@k vs pass^k:** pass@k = P(at least one of k succeeds) — capability, or when a verifier can pick. pass^k = P(all k succeed) = p^k — safety. A 99% gate fails at least once in 10 trials ~10% of the time.

**Statistics (DD §4.7.4):** Wilson interval for 43/50 = **~[74%, 93%]** — "enough to rule out a coin flip, not to distinguish 86 from 80." Paired comparisons → **McNemar** on discordant pairs. Run each task r times at temperature > 0; measure provider non-determinism even at 0. Contamination: public issues may be in training data. **LLM-as-judge** only for fuzzy dimensions, calibrated against ~100 human labels (Cohen's κ), never where an executable oracle exists.

**Evals as deployment gates (POST §32–33):** every behavior axis — code, prompt, model, tool schema, retrieval index, policy — is a deploy and gets the gate. Version the eval set; review deletions of hard cases like code.

**Your Salesforce framework (S4), stated as layers:** retrieval quality (did the right Skill/NLD come back), tool selection, permission enforcement (adversarial cases: user lacks permission → must refuse every time — pass^k), structured outputs, failure paths; **replayed production failures** become golden cases; graceful degradation: low confidence → clarify, tool failure → structured observation to the planner, permission denied → explain + offer approval path, never "retry the LLM until it says yes." Exact suites/CI jobs → **PENDING** (PROBE §9).

**ResolveFlow's two suites (READY):** `eval/harness.py` + `tasks.py` = **11** safety-gate tasks (4 classify, 5 review, 2 execute — fix the README's "12/12", R9); gates deploys in CI. `eval/ capability_harness.py` = 6 capability checks on 3 real issues, non-gating. Feedback loop: `/api/feedback` → `eval/feedback_to_test_cases.py`.

### 3.2 Groundedness & hallucination reduction

**Taxonomy (useful to name):** *intrinsic* hallucination (contradicts the provided context) vs. *extrinsic* (adds claims the context doesn't support); *citation hallucination* (cites a source id that doesn't exist, or one that doesn't support the claim).

**Levers, from cheapest to strongest:**

1. Instruct: cite snippet ids; say "missing_info" rather than guess (ResolveFlow's system prompt).
2. Show relevance scores to the model so it can decline weak citations (ResolveFlow does this).
3. **Structural check on citations** — every cited id must be in the retrieved set *and* above the relevance floor; `bool(citations) and all(...)` so empty citations fail (pinned by `empty_citations_never_ground`).
4. **Entailment check** — per (claim, cited chunk): does the chunk support the sentence? NLI model or a constrained LLM judge ("yes/no"), itself calibrated. **ResolveFlow does not do this yet — F5 / CHANGELOG known limitation**: a relevant chunk can be cited for a claim it doesn't support (the old `taintUniqueValue` case).
5. Corrective retrieval (2.1.4) to remove distractors; better retrieval (rerank, hybrid).
6. Abstention as a first-class outcome; escalation to a human.

**Measuring groundedness:** claim-level support rate (decompose answer into atomic claims; score each against cited context), citation precision/recall. RAGAS-style "faithfulness" metrics do this with an LLM judge — fine if calibrated.

### 3.3 Guardrails — input validation, output sanitization, injection defense

**Input validation (before the model):** schema-validate API inputs (Pydantic / JSON Schema); length caps (cost + DoS); allowlist URL hosts (ResolveFlow only accepts GitHub issue URLs); authenticate and rate-limit (ResolveFlow's per-user daily cap with atomic `INSERT ... ON CONFLICT ... RETURNING count`); **mark untrusted**

**content** — issue bodies, comments, CI logs, retrieved docs are delimited and labeled as data, never merged into instructions.

**Prompt-injection defense (AS §2.5.4, DD §4.11, §7.3):**

- *Direct* injection: the user types "ignore previous instructions." *Indirect*: instructions planted in content the agent reads (issue text, MR description, code comments, CI logs, dependency READMEs, tool outputs, MCP tool descriptions).
- **The lethal trifecta** (GitLab's own threat-model language, from Simon Willison): private data
- untrusted content + ability to act/exfiltrate. Break at least one leg per flow.
- Structural defenses carry the guarantees: tool allowlist derived from the *caller's* authorization, not from anything retrieved; arguments validated in code; reader/writer split; sandbox with no write credential; egress allowlist; human approval of the exact artifact.
- Detectors (GitLab: no checks / log only / interrupt) are signals — "smoke detector, not a firewall" — measure their precision/recall like any classifier.

**Output sanitization (after the model, before anything consumes it):**

- **Schema validation** of structured output; reject or repair, never `eval`.
- **Rendering safety:** model output shown in a web UI or Slack must be escaped (XSS via markdown/HTML; Slack `mrkdwn` link injection like `<https://evil|click here>`); strip or neutralize links to unapproved domains (exfiltration via image URLs that encode data in query strings is a known agent attack).
- **Secret / PII scanning** of outputs and of traces at capture time (DD §4.11 #8).
- **Command/SQL safety:** never splice model text into shell or SQL; parameterize; for `ExecuteSOQL`-style escape hatches, the SOQL comes from a reviewed NLD, not freehand (AS §1.5).
- **Diff-level checks for code agents:** the policy gate (DD §4.13a) is output sanitization for patches — CI config, lockfiles, CODEOWNERS, test deletions.
- **ResolveFlow:** posts the approved comment verbatim — what was approved is what's posted.

**Data governance (resume line):** data minimization (only send the model what the step needs), tenant isolation in retrieval and caches, retention/deletion of traces and checkpoints (traces hold personal data), masking at capture, audit events separate from debug traces (immutable, long-retention vs. short-retention restricted replay artifacts — DD §8.7), provenance (which Skill/prompt/model version produced an action — the execution manifest), and segregation of duties for approvals.

### 3.4 Testing frameworks & code review (resume: "testing frameworks," "code review")

- **Python:** pytest (fixtures, parametrize), ResolveFlow's `eval/tasks.py` dataclass tasks; mock external APIs (GitHub) but call the real LLM in capability evals.
- **Java/Spring:** JUnit 5, Mockito, `@SpringBootTest` (full context) vs. slice tests (`@WebMvcTest`, `@DataJpaTest`), Testcontainers for real Postgres/Kafka in integration tests.
- **JS/TS (Hotstar):** Jest/Vitest, React Testing Library, Playwright/Cypress for e2e; for a player, device-lab testing on real TVs because emulators don't reproduce decoder behavior.
- **Salesforce (AutoRABIT):** Apex test classes, `@isTest`, `Test.startTest()/stopTest()`, 75% org-wide coverage required to deploy to production (Part 8.3).
- **Ruby (GitLab):** RSpec + FactoryBot (DD §11.2).
- **Agent code-review checklist (DD §10.1):** where is authz decided and can the model influence its inputs; what runs twice; idempotency key per side effect; what's in state (no creds, no blobs); budgets; which eval case covers the change (pass^k if safety); span attributes; failure UX; "does any prompt text claim to enforce something — where's the code that does?"

**Self-test (Part 3):**

1. Why must a permission-enforcement eval be pass^k and not an average?
2. Name three output-sanitization controls for an agent that posts to Slack.

3. What's the difference between citation membership and entailment, and which does ResolveFlow check?
4. Where does an injection classifier belong in your architecture, and what does it *not* do?

---

## Part 4 — Distributed systems & deployment

Resume lines: "AWS (Lambda, API Gateway, ECS), Docker, Kubernetes, Spring Boot, REST, GraphQL, microservices, distributed & event-driven systems, system & API design, enterprise architecture patterns"; Salesforce S6 "Deployed and operated agent services on AWS, Docker, Kubernetes, and CI/CD with Splunk observability and event-driven infrastructure — throughput up ~40% with stronger fault isolation; carried on-call rotation ownership"; AdaptiveRAG on Lambda + API Gateway.

> **Read first — this axis lost the CrowdStrike round (2026-09-17).** The concepts below are **READY** (verified against AWS / Kubernetes docs). **Your personal role and your Salesforce configuration specifics are PENDING** until OCQ §A–F comes back. In the room: **state OWN / PATTERN / THIN first, then go one level deeper on the part you owned.** The full infra self-audit table is DD §14.1 (rows I1–I14) — summarized in 4.4.

### 4.1 AWS — Lambda vs. ECS vs. API Gateway, properly

#### 4.1.1 The idea

Three different layers, often confused:

- **API Gateway** is a **front door** (routing, auth, throttling, validation) — not compute.
- **Lambda** is **event-driven function compute** — AWS runs your code per event, scales by running more copies, bills per request × duration.
- **ECS** is a **container orchestrator** — you define tasks (containers) and services (keep N tasks running behind a load balancer); runs on **Fargate** (serverless capacity) or on **EC2** instances you manage.

#### 4.1.2 Lambda mechanics

- **Execution environment lifecycle:** *Init* (download code/container image, start runtime, run your module-level code — imports, DB clients, **loading a FAISS index**) → *Invoke* (handler runs) → environment stays **warm** for reuse → eventually *Shutdown*. A **cold start** = a request that had to wait for Init. Warm invocations reuse globals (so cache clients/indexes at module scope, not in the handler).
- **Limits to know:** max **15-minute** timeout; memory **128 MB–10,240 MB**, with CPU allocated proportionally to memory (more memory = more vCPU, often *faster and cheaper* for CPU-bound work); **6 MB** synchronous request/response payload; deployment package 250 MB unzipped (zip) or **10 GB** container image; `/tmp` ephemeral storage configurable up to 10 GB.
- **Concurrency model:** each execution environment handles **one request at a time**; concurrency = number of environments in flight. Default regional account limit is typically **1,000** concurrent executions (raisable). Each function can scale up by about **1,000 concurrent executions every 10 seconds**. **Reserved concurrency** caps (and guarantees) a function's share; **provisioned concurrency** keeps N environments pre-initialized (no cold starts, billed while idle). **SnapStart** (Java, and since late 2024 Python and .NET) snapshots an initialized environment to cut init time.
- **Invocation types:** synchronous (API Gateway → Lambda; caller waits), asynchronous (S3/SNS/EventBridge; Lambda queues internally, retries twice, then DLQ/destination), **event source mapping** (Lambda polls SQS/Kinesis/Kafka; batch size, partial batch failure reporting).
- **Cold start drivers:** runtime (JVM slower than Python/Node without SnapStart), package size, VPC attachment (much improved since Hyperplane ENIs), and **whatever your init code does** — loading a 500 MB FAISS index from S3 at init can dominate.

- **Connection storms:** 500 concurrent environments × 1 Postgres connection each → exhaust the DB. Use **RDS Proxy** or an external pooler.
- **Newer (verify before citing in depth): Lambda durable functions** (re:Invent 2025) — a checkpoint-and-replay SDK where code re-runs from the start but skips completed checkpointed steps, letting a workflow run up to a year. **This is the same replay-with-checkpoints model as LangGraph's interrupt** — a good comparison point if he asks about durable execution on AWS. Also Lambda **response streaming** for incremental responses.

#### 4.1.3 API Gateway mechanics

- **REST APIs** (feature-rich: usage plans/API keys, request validation, transformations, caching, WAF, private endpoints) vs. **HTTP APIs** (cheaper, lower latency, JWT authorizers, fewer features) vs. **WebSocket APIs** (persistent connections for streaming/chat).
- **Authorizers:** IAM (SigV4), Cognito user pools, **Lambda authorizers** (custom code returns an IAM policy; cache the result by token), JWT authorizers (HTTP APIs).
- **Throttling:** account-level and per-stage/per-method limits (token bucket: rate + burst); usage plans per API key. Returns 429.
- **Timeouts — correct the old fact:** the integration timeout was historically a hard **29 s**. Since **June 2024**, Regional and private REST APIs can request a raise beyond 29 s (may reduce account throttle quota); edge-optimized REST APIs cannot; HTTP APIs max 30 s. **For agent runs of minutes, the right design is still async:** accept → 202 + run id → poll/stream/webhook, not a long-held HTTP request (DD §11.5 uses 202 Accepted).

#### 4.1.4 ECS mechanics

- **Task definition** — the container spec(s): image, CPU/memory, env, secrets (from Secrets Manager/SSM), ports, log config, **task role** (what the app may call in AWS) vs. **execution role** (what ECS needs to pull the image and fetch secrets).
- **Service** — keeps desired count of tasks running; integrates with an **ALB target group** (health checks decide which tasks get traffic); **rolling deployments** (minimum healthy % / maximum %) or blue/green via CodeDeploy; **deployment circuit breaker** can auto-roll back a failing deployment.
- **Service auto scaling** — target tracking on CPU, memory, ALB requests per target, or a custom CloudWatch metric (e.g., **SQS queue depth per task** for workers).
- **Fargate vs. EC2 launch type** — Fargate: no instances to manage, per-task vCPU/memory billing, slower task start (tens of seconds); EC2: cheaper at steady load, GPUs, daemons, more ops.
- **Graceful shutdown:** ECS sends **SIGTERM**, waits `stopTimeout` (default 30 s, max 120 s on Fargate), then SIGKILL. A worker must stop pulling new work and checkpoint within that window.

#### 4.1.5 Which to pick for an agent service (the reasoning he wants)

| Concern | Lambda | ECS / Fargate |
|---|---|---|
| Run duration | ≤ 15 min; agent runs with CI or a human pause must be split into steps (or use durable functions / Step Functions) | Unlimited — but still don't hold a worker across a human pause |
| Traffic shape | Spiky, bursty, scale-to-zero | Steady, high throughput |
| Cost | Per request × duration; expensive at sustained high load | Cheaper per unit at steady load; pay for idle min tasks |
| DB connections | Storms → RDS Proxy | Normal pooling |
| Streaming tokens | Constrained | Natural (SSE/WebSocket behind ALB) |

| Concern | Lambda | ECS / Fargate |
|---|---|---|
| Big dependencies / indexes | Size limits; cold-start pain | Fine |
| Ops | Lowest | Moderate |

**Reference design (DD §14.2):** API Gateway or ALB → thin admission Lambda (authN, validate, dedupe on event id, enqueue, return 202) → **SQS** → **ECS workers** that run graph supersteps and checkpoint to **RDS Postgres** → approvals arrive as API calls that enqueue a resume message. **Lambda at the spiky stateless edges, ECS for the long-running, connection-heavy core.** Step Functions / durable functions as managed durable-execution alternatives — and why you might not: the agent's steps are chosen at runtime by a model (POST §4).

**IAM (Azure-AD-veteran questions):** each Lambda/ECS task has a role with least privilege — read its queue, write its table, call the model endpoint; it **never holds user credentials**; per-user tokens are minted at execution time by a credential broker, scoped per operation. Secrets in Secrets Manager/SSM, not baked into images. Cross-account via STS `AssumeRole` with external id.

#### 4.1.6 Worked example — AdaptiveRAG on Lambda (how to tell it)

"React front end → API Gateway → Lambda (Python) → FAISS index + Claude API. The design question was the index: loading it at module scope during Init means every cold start pays the load, and every concurrent environment holds its own copy in memory — so memory sizing is driven by the index, and provisioned concurrency is the lever if cold starts hurt the latency metric. Because I measured *median* latency, cold starts mostly fall in the tail; I'd report p95 separately." **PENDING:** package type (zip vs. container), memory setting, where the index lives (bundled vs. S3), measured cold start, timeout, how you deployed (SAM/CDK/console). Have these before Oct 9 or say "I'd want to re-check the config."

**Self-test:**

1. What happens during a Lambda cold start, and which part do you control?
2. Why does Lambda + Postgres need RDS Proxy at high concurrency?
3. Is the 29 s API Gateway timeout still a hard limit? What's the right pattern anyway?
4. Task role vs. execution role in ECS.
5. Design the AWS deployment for a durable agent with multi-hour approvals.

### 4.2 Docker & Kubernetes fundamentals

#### 4.2.1 Docker — images and containers

- **Image** = a read-only stack of **layers** (each Dockerfile instruction that changes the filesystem creates one) plus config (entrypoint, env, user). Content-addressed by digest.
- **Container** = a running process (or process tree) from an image, isolated with Linux **namespaces** (pid, net, mnt, uts, ipc, user — what it can *see*) and limited by **cgroups** (CPU, memory — what it can *use*), with a thin writable layer on top (copy-on-write). **A container is a process, not a VM** — it shares the host kernel.
- **Build cache:** a layer is reused if its instruction and inputs are unchanged; once one layer changes, every later layer rebuilds. → Copy dependency manifests and install deps **before** copying source.
- **Multi-stage builds:** build in a fat image, copy only artifacts into a slim runtime image (smaller attack surface, smaller pulls → faster pod start).
- **Tags vs. digests:** tags (`:latest`, `:sha-abc123`) are mutable pointers; digests (`@sha256:...`) are immutable. "Build once, promote the same artifact" = deploy by digest or immutable SHA tag.
- **PID 1 and signals:** the process with PID 1 doesn't get default signal handlers. If `CMD` uses **shell form** (`sh -c "..."`), `sh` is PID 1 and may not forward **SIGTERM** to your app → no graceful shutdown, killed after the grace period. Use **exec form** (`CMD ["uvicorn", ...]`) or an init like `tini`.

- **Security basics:** non-root `USER`, minimal/distroless base, scan images (Trivy, registry scanning), no secrets in layers (they persist in history even if deleted later), `.dockerignore`.

**ResolveFlow's Dockerfile, critiqued (volunteer this — READY):**

```
FROM python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir uv
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev          # good: deps layer cached before source copy
COPY . .
CMD ["sh", "-c", "uv run uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

Good: slim base, lockfile-frozen deps, dependency layer before source. Gaps: (1) **shell-form via `sh -c`** → SIGTERM may not reach uvicorn → in-flight requests cut on redeploy (fix: exec form with a small entrypoint that `exec`s, or `tini`); (2) runs as **root**; (3) no multi-stage; (4) `uv run` at startup may re-resolve — run the venv's uvicorn directly; (5) **and the big one: `render.yaml` uses Render's native Python runtime, so this Dockerfile is not what runs in production** (DD §14.1 I4). Know that before he asks.

### 4.2.2 Kubernetes — the primitives (condensed from DD §14.4, extended)

- **Pod** — smallest schedulable unit: one or more containers sharing network namespace (same IP, localhost) and volumes. Ephemeral; never "fixed," only replaced.
- **Deployment → ReplicaSet → Pods** — declarative desired state; rolling updates governed by `maxSurge` / `maxUnavailable`; `kubectl rollout undo` returns to the previous ReplicaSet.
- **Service** — stable virtual IP + DNS name in front of pods selected by **labels**; kube-proxy routes to *ready* endpoints only. Types: ClusterIP (internal), NodePort, LoadBalancer. **Ingress / Gateway API** for HTTP routing from outside.
- **ConfigMap / Secret** — config and secrets as env vars or mounted files. Secrets are base64, **not encrypted** unless etcd encryption at rest or an external secrets operator (Vault, AWS Secrets Manager via External Secrets) is used. Mounted files update in place; env vars require a restart.
- **Resources:** `requests` drive **scheduling** (a pod lands only where that much is unreserved); `limits` cap usage — exceed memory → **OOMKilled**; exceed CPU → **throttled** (CFS quota), not killed. QoS: Guaranteed (requests = limits), Burstable, BestEffort (evicted first).
- **HPA** — `desiredReplicas = ceil(currentReplicas × currentMetric / targetMetric)`; ~15 s control loop; ~10% tolerance; scale-down stabilization window (default 5 min) to prevent flapping. Metrics: CPU/memory (vs. requests), custom, or external (queue depth via Prometheus adapter / **KEDA**).
- **PodDisruptionBudget** — limits voluntary evictions during node drains/upgrades.
- **StatefulSet** (stable identity + storage — databases), **DaemonSet** (one per node — log agents), **Job/CronJob** (run to completion — re-ingest, eval runs).

### 4.2.3 Probes — and why they matter for an agent mid-execution

| Probe | Failing means | Kubernetes action | For an agent worker |
|---|---|---|---|
| **startupProbe** | Still starting | Hold off liveness/readiness | Model clients, index load, migrations at boot — prevents premature restarts |
| **readinessProbe** | Not ready for traffic | Remove from Service endpoints (no restart) | Fail readiness when draining or when a hard dependency (DB) is unreachable — stop receiving work |
| **livenessProbe** | Wedged | **Restart the container** | Must check the *process* is alive, **not dependencies** |

**The agent-specific trap:** if liveness checks the LLM provider or Postgres, a provider outage makes *every* pod fail liveness → Kubernetes restarts them all → every in-flight agent step is killed mid-execution → work is redone (and non-idempotent side effects may double-fire) → restart storm amplifies the outage. **Rule: liveness = "my event loop responds"; readiness = "I can do useful work right now"; dependencies go in readiness (or nowhere).** Also: an agent worker blocking for 60 s on a model call must not have a liveness timeout that kills it — health endpoints must be served by a thread/loop not blocked by the work.

**Graceful shutdown sequence:** pod marked Terminating → removed from endpoints (asynchronously — add a short `preStop` sleep so in-flight traffic drains) → SIGTERM → app stops pulling new work, finishes or **checkpoints** the current superstep, releases leases → exits before `terminationGracePeriodSeconds` (default 30 s) → else SIGKILL. If killed mid-step, the lease expires and another worker resumes from the last checkpoint — **which is why steps must be idempotent** (DD §14.4).

#### 4.2.4 Worked example — deploying a stateless agent worker on K8s

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: agent-worker}
spec:
  replicas: 3
  strategy: {rollingUpdate: {maxSurge: 1, maxUnavailable: 0}}
  selector: {matchLabels: {app: agent-worker}}
  template:
    metadata: {labels: {app: agent-worker}}
    spec:
      terminationGracePeriodSeconds: 90          # > longest superstep we'll finish before exit
      containers:
      - name: worker
        image: registry.example.com/agent-worker@sha256:...   # immutable digest
        resources:
          requests: {cpu: "500m", memory: "1Gi"}
          limits:   {memory: "1Gi"}               # memory limit; CPU limit often omitted to avoid
throttling
        envFrom: [{configMapRef: {name: agent-config}}]
        env:
        - name: DATABASE_URL
          valueFrom: {secretKeyRef: {name: agent-secrets, key: database_url}}
        startupProbe:   {httpGet: {path: /healthz, port: 8080}, failureThreshold: 30, periodSeconds: 2}
        readinessProbe: {httpGet: {path: /ready,   port: 8080}, periodSeconds: 5}
        livenessProbe:  {httpGet: {path: /healthz, port: 8080}, periodSeconds: 10, timeoutSeconds: 2}
        lifecycle: {preStop: {exec: {command: ["sleep", "5"]}}}
---
# Scale on backlog, not CPU: agent workers are I/O-bound (waiting on models and CI),
# so CPU stays low while the queue grows. KEDA ScaledObject on queue depth / oldest-message age.
```

**Why stateless:** all run state is in the checkpoint store, so any pod can continue any run; worker count scales with *active compute*, not *runs in flight* — a million paused runs need zero pods (DD §6.7).

#### 4.2.5 "Throughput up ~40% with stronger fault isolation" — the mechanism

**PENDING (OCQ F1–F5) — claim only the true one.** Plausible mechanisms (DD §14.4): **bulkheading** (split one shared worker pool into per-workflow / per-downstream deployments + queues, so a slow rate-limited API

saturates only its own pool — throughput rises and fault isolation is the same change seen from the failure side); sync → async queue consumers; right-sizing requests/limits to stop OOMKill/throttle churn; HPA on queue depth instead of CPU. Defensible unit: *completed workflow executions per hour at comparable peak load and resource budget.* Fallback sentence if the method can't be reconstructed: "*I can explain what we changed and why it raised throughput; I'd want to check the measurement window before quoting 40% as more than approximate.*"

**Self-test:**

1. A container vs. a VM in two sentences — namespaces and cgroups.
2. What's wrong with `CMD ["sh","-c","..."]` for graceful shutdown?
3. Liveness vs. readiness for an agent worker — what goes in each, and what's the restart-storm failure?
4. Requests vs. limits: what does each control, and what happens when each is exceeded?
5. Why scale agent workers on queue depth, not CPU?

## 4.3 Spring Boot, REST vs. GraphQL, microservices

**Status:** concepts **READY**. Your Salesforce Java/Spring specifics (which service, which framework version, what you wrote) → **PENDING** — the resume bullet lists "Java, Spring, and TypeScript" for the retrieval-to-action architecture; be precise about which parts were Spring services you wrote vs. platform services you called.

### 4.3.1 What Spring Boot actually provides

Spring Framework is the **dependency-injection (IoC) container** plus a large ecosystem (Web MVC, WebFlux, Data, Security). Spring Boot is **opinionated packaging** on top:

1. **Dependency injection / IoC:** you declare *beans* (`@Component`, `@Service`, `@Repository`, `@Configuration` + `@Bean`); the `ApplicationContext` constructs them and injects dependencies — preferably via **constructor injection** (immutable, testable, no hidden nulls). Why it matters: swap a real `LlmClient` for a fake in tests; one place configures cross-cutting concerns.
2. **Auto-configuration:** `@SpringBootApplication` = `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`. Auto-config classes (listed in `META-INF/spring/...AutoConfiguration.imports` in Boot 3) create beans **conditionally** — `@ConditionalOnClass` (Postgres driver on classpath → configure a `DataSource`), `@ConditionalOnMissingBean` (unless you defined your own). "It just works," and you override by defining your own bean.
3. **Starters:** curated dependency sets (`spring-boot-starter-web`, `-data-jpa`, `-actuator`, `-security`).
4. **Embedded server:** Tomcat (or Jetty/Undertow, or Netty for WebFlux) inside an executable **fat JAR** — `java -jar app.jar`; no external app server; ideal for containers.
5. **Externalized configuration:** `application.yml`, profiles (`dev`, `prod`), env vars override — 12-factor friendly, maps to K8s ConfigMaps/Secrets.
6. **Actuator:** `/actuator/health` (with **liveness and readiness groups** that map directly to K8s probes), `/metrics`, `/info`; **Micrometer** for metrics and, in Boot 3, **Micrometer Tracing** bridged to OpenTelemetry — so W3C trace context propagates through `RestTemplate` / `WebClient` automatically.

**Gotchas a senior engineer knows:** `@Transactional` and other annotations work via **proxies** — a method calling another `@Transactional` method on `this` bypasses the proxy (self-invocation); never hold a DB transaction across an LLM or HTTP call (same rule as DD §11.2's `with_lock`); blocking calls on WebFlux event-loop threads stall everything. Resilience: **Resilience4j** for retry, circuit breaker, bulkhead, rate limiter. His blog post instruments **Java Spring** as well as Rails — expect him to be comfortable here.

### 4.3.2 REST vs. GraphQL

**REST** — resource-oriented: nouns as URLs (`/projects/42/merge_requests/7`), HTTP verbs as operations, status codes as outcomes, HTTP caching (ETag, Cache-Control) for free. Semantics to know: **GET/PUT/DELETE are**

**idempotent, POST is not** (hence `Idempotency-Key` headers for POST creates); 409 Conflict for version mismatches (optimistic concurrency with `If-Match`); 429 with `Retry-After`; 202 Accepted for async work.

**GraphQL** — schema-first: one endpoint; a strongly typed schema (SDL) of types, queries, mutations, subscriptions; the **client specifies the exact shape** it wants; server resolvers fill each field.

| | REST | GraphQL |
|---|---|---|
| Over-fetching | Common (endpoint returns the whole resource) | Solved — ask for fields you need |
| Under-fetching (N round trips) | Common (`/mr` then `/mr/7/pipelines` then …) | Solved — one nested query |
| Caching | HTTP/CDN caching natural | Harder (POST, one URL) → persisted queries, client normalized caches |
| Versioning | URL/header versions | Evolve schema; deprecate fields |
| Server complexity | Low | **N+1 resolver problem** (fix: DataLoader batching), query cost limits (depth/complexity) to prevent abusive queries, field-level authorization |
| Errors | Status codes | HTTP 200 with `errors` array (partial success) |
| Typing / introspection | OpenAPI (optional) | Built-in, introspectable |

**When GraphQL is worth it:** many clients with different data needs over a rich, connected domain (GitLab's UI uses GraphQL heavily alongside REST `/api/v4` — DD §11.2); mobile clients on slow links. **When it isn't:** simple CRUD, service-to-service calls, heavily cacheable public APIs, or when you can't invest in cost limiting and resolver performance.

**For agents specifically:** a GraphQL schema is a self-describing tool surface (introspection), but letting a model compose arbitrary queries is a large, hard-to-authorize surface — prefer **narrow typed tools** (MCP tools / generic CRUD with entity allowlists) that call GraphQL or REST underneath (AS §1.5's generic-vs-bespoke discussion).

#### 4.3.3 Microservices — the core patterns

- **Service boundaries:** split by **business capability / bounded context** (DDD), not by technical layer. A boundary is right when a team can change and deploy the service without coordinating with others. Wrong boundaries produce a **distributed monolith** (lockstep deploys, chatty synchronous calls).
- **Data ownership: database per service**; other services read through its API or events, never its tables. Consequence: no cross-service transactions → **sagas** (sequence of local transactions with compensations; DD §6.8) and **transactional outbox** (write the event to an outbox table in the same transaction as the state change; a relay publishes it → no "state committed but event lost").
- **Communication:** synchronous (REST/gRPC — simple, but couples availability: if B is down, A fails) vs. asynchronous (events/queues — Kafka, SQS, SNS, EventBridge — decoupled, buffered, but eventually consistent and harder to trace). **gRPC** for internal low-latency typed calls (GitLab's Gitaly and Duo Workflow Service are gRPC).
- **Resilience:** timeouts on every call, retries with exponential backoff + jitter **only for idempotent operations**, **circuit breakers** (stop calling a failing dependency; half-open probe), **bulkheads** (separate pools per dependency), rate limiting/backpressure, graceful degradation.
- **Event-driven delivery semantics:** at-least-once is the realistic default → **idempotent consumers** (dedupe on event id), DLQs, ordering only within a partition key (DD §6.1, §6.4).
- **API gateway / BFF**, service discovery, config management, **distributed tracing** (Part 5 — without it microservices are undebuggable).
- **Enterprise architecture patterns (resume line):** strangler fig (incrementally replace a legacy system behind a facade — **this is exactly Agentforce v1→v2 co-existence behind a permission gate**), anti-corruption layer (ResolveFlow's `normalize_evidence` is "the only place that knows GitHub's API shape"), CQRS, event sourcing (and why snapshots beat replay for side effects — 1.4).

**Worked example — when *not* to split:** ResolveFlow is one FastAPI service + graph + Postgres. Splitting `execute` into its own service would add a network hop and a new auth boundary for no team-scaling benefit; the *privilege* separation that matters (only `execute` writes) is already enforced in code. "Boring solutions" (a GitLab value): split when team ownership or scaling profiles diverge, not before.

**Self-test:**

1. What does `@ConditionalOnMissingBean` let you do? What does Actuator give Kubernetes?
2. Over-fetching vs. under-fetching — which does GraphQL solve, and what new problem (N+1) does it create?
3. Why is POST not idempotent, and how do you make a POST create safe to retry?
4. Database-per-service → how do you keep two services consistent without a distributed transaction?
5. Which enterprise pattern describes the Agentforce v1→v2 migration?

### 4.4 CI/CD and the infra honesty audit (condensed; full depth DD §14.1–14.3)

**CI/CD pipeline shape (DD §14.3):** MR → lint/unit/typecheck → build image tagged with git SHA → scan (SAST, deps, container) → integration tests in an ephemeral env → **eval gate** (safety pass^k = 100%; capability no-regression) → merge (merge train) → **build once, promote the same artifact** → staging + synthetic agent run → canary 1–5% compared on **error/latency *and* online eval metrics** → ramp → automatic rollback (previous image / ReplicaSet; feature flag off for behavior changes). DB migrations expand → migrate → contract so the old version runs on the new schema; paused runs checkpointed by v2 must be resumable by v1 or drained.

**GitHub Actions vs. Jenkins vs. GitLab CI** — table in DD §14.3. Jenkins (your AutoRABIT era, **PENDING** what you built): controller + agents you operate, `Jenkinsfile` declarative/scripted Groovy pipelines, credentials binding plugin, shared libraries.

**Your public OWN example (READY):** ResolveFlow `.github/workflows/deploy.yml` — `verify` runs the safety-gate harness; `deploy-frontend` (Vercel) and `deploy-backend` (Render deploy hook) both `needs: verify`; weekly `reingest.yml`; `keep-warm.yml`. **Volunteer the gaps:** no unit tests job, no staging, no canary, manual rollback, deploy jobs `exit 0` when secrets are missing (a deploy that "succeeds" without deploying), Render hook key in a URL query string, and the hook deploys the *branch head*, not the SHA that passed `verify` (race on close pushes).

**Infra self-audit summary (DD §14.1 — fill the Rating column via OCQ):**

| # | Claim | Public evidence | Status |
|---|---|---|---|
| I1 | AWS for agent services | ResolveFlow is Render + Vercel + Postgres + Pinecone, **not AWS**; AdaptiveRAG = Lambda + API GW | **PENDING** (OCQ A1–A8) |
| I2 | Lambda / API Gateway | AdaptiveRAG | **PENDING** config specifics |
| I3 | ECS | none public | **PENDING** |
| I4 | Docker | ResolveFlow Dockerfile — not what runs on Render | READY (with that caveat) |
| I5 | Kubernetes | none public | **PENDING** (OCQ C1–C8) |
| I6 | GitHub Actions | ResolveFlow deploy.yml | **READY — OWN** |
| I7 | Jenkins | none public | **PENDING** (OCQ B1–B3, B8) |
| I8 | Event-driven infra | none public | **PENDING** (OCQ A6) |
| I9 | Splunk | none public | **PENDING** (OCQ D1–D7) |
| I10 | Distributed tracing | none public | **PENDING** (OCQ D2) |
| I11 | ~40% throughput | internal | **PENDING** (OCQ F1–F5) |

| # | Claim | Public evidence | Status |
|---|---|---|---|
| I12 | On-call | internal | **PENDING** (OCQ F6–F9) |
| I13 | Data Cloud | AS §1.4 / DD §14.6 (product-level) | **PENDING** consumed vs. configured (OCQ E) |
| I14 | AutoRABIT +30% | internal | **PENDING** (OCQ B8) — Part 8.3 gives the domain depth |

**Fallback sentence for any THIN row:** "*I haven't run that directly — at Salesforce the platform team owned it and I deployed onto it. Here's how my service used it, and how I'd reason about your question from first principles.*" Then use 4.1–4.3. **Never answer an infra question with agent-architecture material to change the subject** (DD §14.7).

---

## Part 5 — Observability & distributed tracing

Resume lines: "observability tools (Splunk), performance monitoring"; "debugging failures across distributed tracing, structured logs, and CI signals"; "model performance monitoring"; Bug Triage (Splunk correlation). **Status:** OTel concepts **READY** (verified); your Splunk indexes, SPL, dashboards, real incident → **PENDING** (OCQ D1–D7). His blog-post pattern and the agent attribute schema are in DD §0.1, §4.8, §14.5 — condensed in 5.5, not re-derived.

### 5.1 The idea, slowly

A request in a distributed system touches many processes. Logs from each are separate files; a metric says *that* p95 went up, not *where*. **Distributed tracing** reconstructs one request's path as a tree of timed operations across processes, so you can see which hop was slow or failed. **OpenTelemetry (OTel)** is the vendor-neutral standard (CNCF) for producing traces, metrics and logs: an API, SDKs per language, semantic conventions (standard attribute names), the **OTLP** wire protocol, and the **Collector** (receive → process → export to any backend — Jaeger, Tempo, New Relic, Splunk Observability, Datadog). Instrument once, change backends without re-instrumenting — which is precisely why a New Relic architect cares.

### 5.2 What a span actually contains

A **trace** is a tree (DAG) of **spans** sharing a `trace_id`. A span is one timed operation:

| Field | Meaning |
|---|---|
| `trace_id` | 16 bytes (32 hex) — same for every span in the request |
| `span_id` | 8 bytes (16 hex) — this operation |
| `parent_span_id` | The caller's span (empty for the root) |
| `name` | Low-cardinality operation name: `POST /api/resume`, `node:await_approval`, `tool:post_comment` |
| `kind` | `SERVER`, `CLIENT`, `INTERNAL`, `PRODUCER`, `CONSUMER` |
| start / end timestamps | Duration |
| `attributes` | Key-values: `http.request.method`, `db.system`, `tenant.id`, `gen_ai.request.model` |
| `events` | Timestamped annotations inside the span (e.g., exception with stack trace, "retry scheduled") |
| `links` | References to spans in **other** traces (batch consumers; **resume after a long pause**) |
| `status` | `Unset` / `Ok` / `Error` (+ description) |
| `resource` | Who emitted it: `service.name`, `service.version`, `deployment.environment`, k8s pod |

**Span context** = `trace_id` + `span_id` + `trace_flags` (sampled bit) + `trace_state` — the part that crosses process boundaries.

### 5.3 Context propagation — in-process, across processes, across async hops

**Across processes (HTTP/gRPC):** W3C Trace Context header

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
             ver  trace-id (32 hex)                parent span-id    flags (01 = sampled)
tracestate:  vendor-specific key=values
baggage:     tenant.id=acme,flow=fix_ci        # app-level key-values that travel with the context
```

The client's instrumentation **injects** headers; the server **extracts** them and starts a child span. Auto-instrumentation libraries (requests, httpx, FastAPI, Spring via Micrometer Tracing) do this for you.

**Within a process:** the "current span" lives in a context object. Python uses **`contextvars`**; Java uses thread-locals (with context wrapping for executors); Node uses AsyncLocalStorage. **Where traces break in Python** (worth knowing because LangGraph runs sync nodes in a thread pool under `ainvoke`): contextvars flow into `asyncio` tasks (copied at creation) and into `asyncio.to_thread`, but **not** into a bare `ThreadPoolExecutor.submit` — you must capture and attach the context explicitly (`contextvars.copy_context().run(...)` or OTel's context API). Symptom: spans from those nodes appear as orphaned root spans.

**Across async hops (queues, Sidekiq, Kafka, SQS):** there are no HTTP headers, so put `traceparent` in **message metadata** (Kafka headers, SQS message attributes, Sidekiq job args). Producer span (`PRODUCER`) → consumer span (`CONSUMER`) as child — or as a **link** when one consumer batch processes messages from many traces.

**Across a durable human pause (the agent-specific case):** the approval may come hours later in a different process; the parent span ended long ago. Persist the trace context **in the checkpoint**, and on resume start a new span **linked** to the original (span link) and carrying the same correlation attributes — so "follow the `traceId`" still works across the pause. **MCP's 2026-07-28 revision now documents carrying `traceparent`/`tracestate`/`baggage` in request `_meta`** — so trace context can flow host → MCP server too.

**Baggage caution:** baggage propagates to *every* downstream, including third parties; never put PII or secrets in it. Put `tenant.id` in baggage only if all downstreams are yours; otherwise set it as a span attribute per service.

### 5.4 Sampling

- **Head sampling** — decide at the root (e.g., `ParentBased(TraceIdRatioBased(0.1))` keeps 10%); the decision travels in the `sampled` flag so every service agrees. Cheap; but you decide before knowing whether the request will fail.
- **Tail sampling** — the Collector buffers whole traces and decides after they complete (`tail_sampling` processor): keep all errors, all slow traces, a % of the rest. Costs Collector memory and requires all spans of a trace to reach the same Collector instance (load-balancing exporter keyed by trace id).
- **For agents:** keep **100% of traces with a policy deny, an escalation, an error, a safety-gate failure, or cost above p99**; sample routine successes. Agent runs are low-volume compared to HTTP requests, so keeping most agent traces is often affordable — but redact prompt/response payloads at capture.

### 5.5 His correlation-attribute pattern, applied to agents (condensed — DD §4.8, §14.5)

His post: propagate `traceId`, `upstreamApplication`, `userId`, `tenantId` across every hop; find the root cause by following one `traceId` (`FROM Transaction SELECT traceId, appName, upstreamApplication`). The agent version — every span carries:

`traceId` (one agent run, across the pause) · `upstreamTrigger` (his `upstreamApplication`: the pipeline id / finding id / upstream agent's traceId) · `tenantId`, `projectId` · `requestingUserId`, `actingIdentity` (composite identity) · `threadId`, `checkpointId`, `node` · execution manifest: `flowVersion`, `promptVersion`, `modelId`, `toolsetVersion`, `policyVersion`, `indexVersion` · `tool.name`, `tool.args_hash`, `policy.decision`, `policy.rule` · `tokens.in/out`, `cost.usd`, `iteration`.

**OTel GenAI semantic conventions** (in development; now maintained in a dedicated repo) define standard names for model and agent spans — e.g., `gen_ai.operation.name`, `gen_ai.provider.name`, `gen_ai.request.model`, `gen_ai.usage.input_tokens` / `output_tokens`, and span types for `invoke_agent` and `execute_tool`. Mention them; don't recite.

**High-cardinality discipline:** `tenantId`, `userId`, `traceId` go on spans and logs, **never on metric labels** (cardinality explosion). Metrics: low-cardinality dimensions (flow, model, outcome, tier).

**The four signals (DD §14.5):** structured logs (forensics; Splunk), traces (causality + latency, across async hops and the pause), metrics (SLOs, burn-rate alerts), CI signals (change markers — "what changed?" is the first on-call question).

**The worked on-call trace to rehearse (DD §14.5):** "Slack approval succeeded but the change never applied" → SPL from `approval_id` to `traceId` → follow the trace across slack-gateway → orchestrator → action-worker → auth-broker → root cause: refresh token revoked between request and approval; 401 treated as retryable; success message posted on *enqueue* not *execution*. Two fixes, one per layer, plus a regression case. **Replace illustrative names with your real ones once OCQ D6 returns.**

### 5.6 Splunk specifics (what to be fluent in)

SPL shape: `index=agent_svc sourcetype=... traceId=<id> | table _time service node status latency_ms | sort _time`; `stats count by status, workflow`; `timechart span=5m p95(latency_ms) by workflow`; `transaction traceId` (group events into a transaction — expensive; prefer `stats`). Structured JSON logs with consistent field names make this work; free-text logs don't. Saved searches → alerts; dashboards per workflow. **AI Bug Triage (hackathon)** automated exactly this move: a Claude Code skill pulled Splunk logs for the time window, correlated with `git log`/blame, recent changes and the work item, and wrote a structured report with cited log lines — "the `traceId`-following move from his blog post, automated" (DD §2.5). 90 → 15 min is time to assemble diagnostic context, **not** time to fix; know n (**PENDING**).

**Self-test:**

1. List the fields of a span. Which ones cross the process boundary?
2. Write a `traceparent` header and label its parts.
3. Where do Python traces break in a thread pool, and how do you fix it?
4. How does a trace survive a six-hour human approval pause?
5. Head vs. tail sampling — which would you use for agent runs and why?
6. Why must `tenantId` never be a metric label?

---

## Part 6 — Agentforce for Setup v1 → v2 (condensed)

**Full depth:** PROBE §0 (verified component model), AS §1 (architecture at full depth), HLD (primary source, Jan 16 2026), SG-GL §2 (funnel taught), SG-GL §5 (Data Cloud taught), DD §14.6 (Data Cloud numbers). **Status:** architecture **READY** (HLD-sourced). **Your personal share** of the migration, NLD counts, latency per hop, which components you built → **PENDING** (PROBE §1–3, §11).

### 6.1 The pitch (AS §1.1)

v1 (Agentforce 2.0): topics + ~45 bespoke Java actions + a ReAct planner. v2 targets the **long tail** — "the 30–40% of use cases that represent over 90% of user needs but are too numerous to" hard-code (HLD). Three pillars: **stateful Agent Graph** (Agentforce 3.0 — Daisy + AgentGraph DSL), **multi-layered Knowledge Base** (Data Cloud), **Generic Actions**. One non-negotiable principle: a hard boundary between non-deterministic *thinking* and deterministic *doing*, with a human in the loop before any modification executes.

### 6.2 Component model (PROBE §0)

```
user ─► ConversationManager (Supervisor; owns state + all user comms; control ALWAYS returns here)
          │ TriageRequestIntent → INFORMATION_SEEKING | ACTION_SEEKING
          ├─► Knowledge Node: product docs, NLDs, Metadata & API grounding against the LIVE org
          ├─► Reasoner Node:  High-Velocity Path (one Workflow NLD → translate to executionPlan)
          │                   Dynamic Synthesis (multi-NLD merge | Higher-Order "UBER plan" | hybrid)
          │     Plan Generation Pipeline:
          │       1 GatherMoreDetails (clarify vs NLD requirements; Consultative Dialogue Loop)
          │       2 Scaffold Ingestion (deterministic: NLD `steps` array)
          │       3 compilePlan: Stage A parallel vector search per step → 3–5 candidates
          │                      Stage B one grounded LLM call → JSON executionPlan
          │                      Stage C parse + schema-validate (+ optional pre-flight, confidence)
          └─► Executor Node:  NO LLM, 100% fidelity, resolves data dependencies between steps
```

**Generic Actions:** Discovery (unified search; `ExecuteSOQL` escape hatch with NLD-authored SOQL); Context/Grounding (`QueryDataSource` replacing v1's `HelpNavigationAction`, `APIGrounding`, `TenantGrounding`); Generic CRUD `ExecuteEntityCreate/Read/Update/Delete` (planner supplies API family Data vs. Tooling + Entity API name); Display (incl. Generative UI from an allowlist of existing components). **NLD = Knowledge as Code** — Markdown prose + JSON header (`entities`, `operations`, `permissions`, `relatedDocuments`, `scopes`, `tools`); e.g., `clone_user.md`. Adding a capability = authoring a reviewed NLD, not Java. NLDs are LLM-drafted then **human-approved** before ingestion.

**Metadata API vs. Tooling API (AS §1.5):** Metadata API deploys/retrieves configuration, often async and file-based, for bulk config; Tooling API gives fine-grained, synchronous, SOQL-queryable access to metadata records for developer tooling. (Part 8.3 uses the Metadata API from the release-management side.)

### 6.3 V1 → V2: the reasons, paired one-to-one (QB §3)

| v1 limit | v2 answer |
|---|---|
| Limited ReAct planner, no real reasoning step | Stateful Agent Graph with a formal plan-generation pipeline |
| No clarification — always picks the most-likely action | `GatherMoreDetails` + Consultative Dialogue Loop |
| LLM topic selection doesn't scale | No topics; triage + retrieval |
| Action selection doesn't scale past ~45 actions | Vector-search action discovery (the funnel) |
| New capability = new Java | Knowledge as Code (NLDs) + generic CRUD |

**Migration (HLD):** (1) **co-existence** — v2 alongside v1 behind a **permission gate**; (2) **content migration** — v1 Topics/Instructions → v2 NLDs; (3) **action consolidation** — bespoke actions supported initially, retired toward generic ones (`UpdateUserRecordFields` → `ExecuteEntityUpdate`). That is the **strangler-fig** pattern (4.3.3) — say the name.

### 6.4 The funnel, in one breath (SG-GL §2)

"For each scaffold step, a parallel vector search narrows hundreds of actions to 3–5 candidates; one grounded LLM call picks per step and emits the plan. The thing that scales with the catalog (vector search) is cheap and parallel; the expensive thing (the LLM) never sees more than five options per step." Failure mode: Stage A recall miss → measure recall@5 per catalog size. Why per-step search: different steps need semantically different actions; merging dilutes the query.

### 6.5 Data Cloud retrieval (SG-GL §5, DD §14.6)

Chain: `QueryDataSource` → Org CS (Connected App/ECA) → custom **TokenExchange** handler mints a tenant-scoped **`orgJWT`** → **SFAP AI Gateway** → Data Cloud (DC1/UKD): **Unified Search Index** for product/help docs (Help Agent Retriever) and a separate **NLD Search Index** (mDMO from mDLOs) → back through the gateway. NLDs provisioned per org (**Metadata DLO Provisioning BT Action**), kept current by the **Metadata Extraction BT Action**: `New → Processing → Syncing → Data_Copied → Ingestion_Requested → Ingestion_Complete`. Optimization: NLDs served from Core's classpath to skip a C360 round trip. Isolation is **structural** (the retriever never holds a credential for another org) — RFC 8693-style token exchange, his IAM home turf. Data 360 is "a derived, eventually-consistent data plane": CRM ingestion ~10–15 min, identity resolution 1–24 h, ACL filter fields only as fresh as the last re-embed → live grounding for anything permission-sensitive. **Your consumed-vs-configured answer → PENDING (OCQ E8).**

### 6.6 Disclosed risks (HLD, Jan 2026 — confirm current status, PROBE §6)

- Centralized C360 Data Cloud retrieval pipeline "still uncertain, having limited scalability."
- AgentGraph **10–20% slower** than the ReAct planner (new thinking phase); bet: Daisy++.
- AgentGraph parity gaps and bugs; DSL file-based agent support delayed.
- Proposed (not built): **Parallel Action Execution** — `execution_mode: "parallel"` per plan stage for independent steps after a shared dependency.

**Volunteer the latency trade-off as self-awareness** (QB §2): "The stateful planner is measurably slower per request; the bet is reliability on the long tail. I'd rather state that than pretend it's free."

### 6.7 Honest gaps at Salesforce (ART §7–8)

Session persistence / crash recovery and compaction were not shipped in v2 per the source docs — **HONEST GAP at Salesforce** (verify nothing shipped since: PROBE §5). Your ResolveFlow durable persistence is the shipped counterpart (1.1.6).

**Self-test:** draw the component model from memory; give the v1→v2 pairs; explain why tenant isolation happens before retrieval; name the three HLD risks; say what "control returns to the CM" buys.

---

## Part 7 — ResolveFlow, end to end (condensed)

**Full depth:** DD §2.3 (code-level), §2.4 (findings F1–F9), §3.4–3.5 (Stories D, E), DD §1.9 (skeleton), repo `~/Desktop/ResolveFlow`. **Status: READY** except rows flagged below.

### 7.1 Architecture in one diagram

```
 Vercel (frontend) ──/api/* proxy (first-party cookie)──▶ FastAPI on Render (app/main.py)
                                                          │  GitHub OAuth (server-side state, 10-min TTL,
single-use)
                                                          │  sessions (opaque httpOnly cookie),
```




### 7.2 The decisions you defend

- **Rule-based `classify`:** the router allocates autonomy, so it must be cheap, fast, deterministic. Evolution: LLM *proposer* with a rule *floor* — the model can never route *up* in autonomy past what rules allow.
- **Model reasons, code decides:** `outcome = "approve" if (groundedness_ok and risk_ok and permission_ok) else "escalate_to_human"` — "the judge explains; code decides."
- **Every path to `execute` goes through `await_approval`**, and `execute` re-checks anyway.
- **Partial success reported, not hidden:** label failure after a posted comment is captured as `label_error`.
- **Empirical heuristic fix:** `is_information_sparse` matched 0 of ~380 real issues (templates pad bodies; bots comment) → strip template noise; re-verified ~2% fire rate, zero false positives on the negative control.
- **IAM details:** server-side OAuth `state` consumed with `DELETE ... RETURNING` (atomic single-use + TTL); `httponly/secure/samesite=lax`; `/api/resume` 403 unless you own the thread (BOLA fix); atomic per-user daily cap; pooled connections pinged (`check_connection`) because serverless Postgres kills idle ones.
- **CI gates deploys on the safety suite** (I6).

### 7.3 Findings to volunteer (DD §2.4) — each with its fix

| # | Finding | Fix |
|---|---|---|
| F1 | Concurrent resume can double-post | Per-thread advisory lock / lease + `executed_actions` unique key |
| F2 | Pre-`interrupt()` code runs twice | Keep it pure; hash the payload; require resume to echo the hash |
| F3 | GitHub writes not idempotent across crashes | Marker `<!-- resolveflow:{thread}:{checkpoint} -->` + read-before-write |
| F4 | `public_repo` scope, plaintext token, shared-token fallback | GitHub App, fine-grained `issues:write`, 1-h installation tokens, envelope encryption, no fallback for writes |
| F5 | Groundedness ≠ entailment | NLI / calibrated yes-no judge per (claim, chunk) |
| F6 | No run-level deadline | Per-run deadline propagated; explicit LLM timeouts / `max_tokens` |
| F7 | Approvals never expire | TTL + re-validate evidence freshness at resume |
| F8 | `thread_owners` written after run | Write owner row before invoking |
| F9 | Resume claims vs. code | The audit table below |
| (new) | `github_token` checkpointed inside state | Store `credential_ref`; resolve at execution |

### 7.4 The honesty audit (DD §0.3, flags intact)

| # | Claim | Evidence | Status |
|---|---|---|---|
| R1 | LangGraph agent diagnosing GitHub issues | `graph/build.py`, 7 nodes | **READY** |
| R2 | ~2,750 closed-issue chunks | `ingest.py`: 4 × 150 issues, 800/40, Pinecone | **READY** |
| R3 | Measured relevance thresholds gate evidence | `MIN_RELEVANCE_SCORE = 0.35`, CHANGELOG calibration | **READY** |
| R4 | "repository context, closed issues, PRs, code evidence, CI signals" | Issues, comments, CODEOWNERS, linked PR, check-runs ✅; **source files not fetched** (`files_touched=[]`); `owner_hint` unused in prompt | **PENDING** — wire it or say precisely "code evidence = CI on the linked PR's head SHA + CODEOWNERS" |
| R5 | Independent critique; deterministic approve/escalate | `independent_review.py` | **READY** |
| R6 | Interrupts + Postgres durable pause/resume | `await_approval.py`, `AsyncPostgresSaver` | **READY** |
| R7 | OAuth; writes refuse without authorization | `execute.py` `PermissionError`; token threaded | **READY** with F4 nuances volunteered |
| R8 | 50-scenario benchmark, OpenAI/Anthropic/open-weight, 86% / 24% | **Not in repo**; only `gpt-4o-mini`; `classify` is rule-based | **HIGHEST RISK — PENDING**: commit it, build it (DD §4.7.6), or say it first |
| R9 | README "12/12 passing" | harness = **11** tasks | **PENDING** — fix the README |
| R10 | CHANGELOG `DAILY_ANALYZE_LIMIT = 10` | `app/db.py` = 30 | **PENDING** — align |
| R11 | Comment: 0.35 favors escalation | 0.35 < midpoint 0.375/0.405 → leans to approve | **PENDING** — fix comment or raise τ |

**Rule for the room:** "a number you can't derive the method for is a liability, not an asset."

### 7.5 Story D and Story E in 30 seconds each

- **D — the vanished approval:** `SqliteSaver` on Render's ephemeral disk lost paused runs on container recycle → `AsyncPostgresSaver`; went fully async at the API (`ainvoke`, `await saver.setup()`), nodes stayed sync; then `AdminShutdown` from serverless Postgres → connection check. Lesson: "durability is a property of where the bytes live, not of the API you call."
- **E — the relevance gate:** generic diagnoses passed because the gate only checked id membership → surfaced cosine scores, measured clusters, set the floor, showed scores to the model, added `weakly_related_citation_escalates`; grew corpus 40 → 600 issues; `taintUniqueValue` went from wrong to right. Weakness: calibrated on few points → PR curve on ~200 labeled pairs.

---

## Part 8 — Disney+ Hotstar, MakeMyTrip, AutoRABIT — explained for the room

Why these matter to *him*: each is an instance of a pattern he cares about. Hotstar = **telemetry-driven control loop + graceful degradation at extreme scale** (observability). MMT = **idempotent integrations + compliance-as-code** (correctness under external failure, identity verification). AutoRABIT = **CI/CD and release tooling** (GitLab's core product). Rule for all three: describe the **mechanism**, not the bullet. Where the exact mechanism you built

isn't in a document, this part gives the *plausible technical mechanisms* — **pick the one that is true and say only that** (status **PENDING** on your specifics; concepts **READY**).

## 8.1 Disney+ Hotstar (SDE II, Jan 2023 – Dec 2024)

Bullets: "Architected high-scale streaming experiences using React, Next.js, JavaScript, and Shaka Player across web and Living Room platforms"; "Reduced player failure rate ~70% with a runtime mechanism that identified and dynamically blacklisted unreliable decoding layers — graceful degradation under real-world failure conditions"; "Contributed to Shaka Player (open source), resolving timestamp-handling defects that eliminated stream repetition during World Cup playback; built Web Worker-based playback reporting for real-time error collection and faster incident investigation."

**Scale context (public, verified):** during the ICC Men's Cricket World Cup 2023, Disney+ Hotstar set its peak-concurrency record five times, peaking at **~5.9 crore (~59 million) concurrent viewers** for the India–Australia final on Nov 19, 2023. You can say: "the World Cup final peaked around 59 million concurrent streams on the platform — at that scale a 0.1% player failure rate is tens of thousands of broken sessions."

### 8.1.1 Adaptive streaming and what Shaka Player is

**Adaptive bitrate (ABR) streaming:** the video is encoded at several bitrates/resolutions ("renditions"), each cut into short **segments** (typically 2–6 s). A **manifest** lists them — **DASH** (`.mpd`, XML) or **HLS** (`.m3u8` playlists). The player downloads segments over plain HTTP from a CDN and switches rendition per segment as bandwidth changes. For **live**, the manifest is refreshed periodically and the player stays near the live edge.

**Browser plumbing: MSE (Media Source Extensions)** lets JavaScript feed media bytes into a `<video>` element via `MediaSource` + `SourceBuffer.appendBuffer()`; **EME (Encrypted Media Extensions)** lets JS negotiate DRM licenses (Widevine, PlayReady, FairPlay) with the browser's Content Decryption Module.

**Shaka Player** is Google's open-source JavaScript library that implements DASH and HLS playback on top of MSE/EME. Its architecture (verified against the Shaka docs):

| Component | Job |
|---|---|
| `shaka.Player` | Public API; orchestrates everything |
| Manifest parsers (`DashParser`, `HlsParser`) | Turn the manifest into Shaka's internal model: periods → **variants** (audio+video stream pairs) → segment references with start/end times |
| `NetworkingEngine` | All HTTP requests; request/response filters (auth tokens, CDN signing), retries with backoff |
| `AbrManager` (`SimpleAbrManager` + EWMA bandwidth estimator) | Chooses the variant from measured throughput |
| `StreamingEngine` | Per stream, decides *which* segment to fetch next to keep the buffer at its goal; handles switches, live manifest updates |
| `MediaSourceEngine` | Appends segments to `SourceBuffer`s (transmuxing TS → MP4 where needed), sets `timestampOffset` and append windows |
| `DrmEngine` | License acquisition via EME |
| `TextEngine` | Subtitles/captions |
| Playhead / gap jumping | Tracks position, jumps small buffered gaps, handles stalls |

Errors surface as `shaka.util.Error` with **category** (NETWORK, MEDIA, MANIFEST, DRM, STREAMING…) and **code** (e.g., decode-related MEDIA errors such as 3016 for a `<video>` error) and severity (RECOVERABLE vs CRITICAL).

### 8.1.2 "Timestamp-handling defects that caused stream repetition" — what it plausibly means

Every segment carries **media timestamps** (PTS/DTS; in fMP4 the `tfdt` box's `baseMediaDecodeTime`). The player maintains a **presentation timeline**, and the mapping from "segment N" to "seconds on the timeline" comes from the manifest (DASH `SegmentTimeline` `t/d`, `presentationTimeOffset`; HLS media sequence numbers, `EXT-X-DISCONTINUITY`, `EXT-X-PROGRAM-DATE-TIME`). The player uses `SourceBuffer.timestampOffset` to place each segment's media time at the right presentation time. **If the player's idea of a segment's time disagrees with the media's own timestamps, two things go wrong:** the StreamingEngine thinks the buffer ends earlier than it really does and **re-fetches a segment it already appended**, or a segment is appended at the wrong offset so the same content plays again → the viewer sees the **same few seconds repeat**. Known families of such bugs in live HLS/DASH players (Shaka's own issue tracker has "HLS live stream looping" and timestamp-offset-on-variant-switch reports):

1. **Discontinuity handling** — at an ad break or encoder restart, timestamps reset; if the offset for the new discontinuity sequence isn't applied, segments land at the wrong time.
2. **Variant switch during live** — after switching rendition, offsets computed from the *old* stream's media times are applied to the new one → misalignment → re-fetch/loop.
3. **Manifest-time vs. media-time drift** — live manifests that slide forward; rounding in timescale arithmetic (`t / timescale`) accumulating error over a multi-hour match.
4. **MPEG-TS PTS rollover** — PTS is a 33-bit counter at 90 kHz, so it wraps every 2^33 / 90,000 ≈ **26.5 hours**; a long-running live channel crosses it, timestamps jump backward, and naive arithmetic thinks the new segment is *earlier*.

**How to tell it in the room:** "The bug was in how the player mapped segment media timestamps to the presentation timeline in [the specific case you fixed — e.g., across a discontinuity / after a variant switch / on manifest refresh]. The symptom was that the StreamingEngine re-requested content it had already buffered, so viewers saw a few seconds repeat. The fix was [the specific change], with a test reproducing the timestamp sequence from a captured live manifest." **PENDING: find your actual PR on github.com/shaka-project/shaka-player (search your GitHub handle), and quote its title and the root cause exactly** — he can check it in 30 seconds; it's also your only public, verifiable open-source contribution, so it's an asset if you know it cold.

### 8.1.3 "Dynamically blacklisting unreliable decoding layers" — the mechanism

**The problem:** "Living Room" = smart TVs (Tizen, webOS), Android TV, set-top boxes, consoles — thousands of device/firmware combinations with **hardware decoders** of varying quality. A device can *claim* support (`MediaSource.isTypeSupported('video/mp4; codecs="hev1..."')` or `navigator.mediaCapabilities.decodingInfo()` returns `supported: true`) yet fail in practice for a specific **codec / profile / level / resolution / frame-rate / HDR** combination: decode errors, green frames, freezes, audio passthrough failures (e.g., Dolby/E-AC-3 on certain soundbars), or DRM + hardware-decoder interactions. Capability APIs answer "is it *supported*," not "is it *reliable on this firmware*."

**"Decoding layer"** = the decode path a variant forces: codec (H.264 vs HEVC vs AV1), profile (Main vs Main10), resolution/frame rate, hardware vs software decode, secure (DRM) vs clear path.

**The mechanism (generic shape — confirm your actual one):**

1. **Detect** — classify runtime errors by cause: decode/MEDIA errors and stall patterns attributed to the *active variant's* codec/profile, keyed by device model + firmware + browser/OS version. Distinguish from network errors (those aren't the decoder's fault).
2. **Mitigate in-session** — remove the failing decoding path from ABR's choices and switch to a known-good one (e.g., HEVC → H.264, 1080p50 → 720p). Shaka supports this shape natively in recent versions: restrictions / preferred codecs, and temporary variant disabling (`disabledUntilTime`, `streaming.maxDisabledTime`, default 30 s) after errors.

3. **Remember** — persist the blocklist entry on the device (localStorage/IndexedDB) so the next session starts on the safe path, **with a TTL** so a firmware update gets a second chance.
4. **Fleet-level** — aggregate telemetry by device class server-side; when a combination's failure rate crosses a threshold, push it into a **remote config** blocklist that all devices of that class fetch at startup, before playback begins.

**Name it in his language:** "a per-decoder-path **circuit breaker**, keyed by device class, driven by telemetry — an observability-driven control loop: signal → attribution by device → automated mitigation → verify the failure rate dropped." That's the 1.10 closed loop in DD minus the LLM (Story G, DD §3.7).

**The ~70% number — PENDING (DD §2.1 template):** failure rate = sessions (or playback attempts) ending in a fatal player error ÷ sessions started? Over which devices — all, or Living Room? Which window (before/after rollout), and was World Cup traffic in one window and not the other? What did the remaining 30% consist of (network/CDN failures the decoder fix can't touch)?

**Trade-off to volunteer:** blocklisting a decoder path trades **quality for reliability** (an HEVC-capable TV forced to H.264 uses more bandwidth for the same quality, or drops a resolution); a false-positive blocklist entry silently degrades a whole device class — hence TTLs and per-entry hit/recovery metrics.

### 8.1.4 Web Worker–based playback reporting — why off the main thread

**What a Web Worker is:** a script running on a **separate OS thread** with its own event loop, created with `new Worker(url)`. It has **no DOM access**; it communicates with the page only by `postMessage` (data is copied via the **structured clone** algorithm, or **transferred** zero-copy for `ArrayBuffer`s). Workers can use `fetch`, timers, IndexedDB, compression streams.

**Why it matters for a video player:** in the browser, the **main thread** runs JavaScript, style, layout, paint coordination, input handling — *and* the player's own JS (manifest parsing, ABR logic, MSE append scheduling, ad logic, React reconciliation for the UI). On a low-power TV SoC the main thread is the scarce resource. Telemetry work — building payloads with stack traces, buffered ranges, network timing, serializing JSON, batching, compressing, retrying failed uploads — is pure overhead on that thread. Done on the main thread it causes **long tasks (> 50 ms)**: remote-control input lag, UI jank, and in the worst case **delayed segment appends → buffer underrun → the very stall you're trying to report.** A reporter that causes the failure it measures is the observer effect at its worst.

**Design (generic — confirm yours):** the player emits small events (`postMessage({type, ts, sessionId, variant, error})`) → the worker enriches, batches (every N events or T seconds), compresses, sends with retry + backoff, and queues in IndexedDB when offline → backend indexes by **session id, device class, CDN, variant, error code** → real-time dashboards and alerting. Caveat to volunteer: a worker dies with the page, so the final flush on `pagehide`/`visibilitychange` happens from the main thread (`navigator.sendBeacon` lives on the page's navigator, or `fetch` with `keepalive`).

**"Faster incident investigation":** with every session's errors keyed by a session id and device/CDN/variant attributes, "why are Samsung 2019 TVs failing in Mumbai on the 1080p variant?" is a query, not a war room. **Bridge to him:** the session id is a `traceId`; device/CDN/variant are his correlation attributes; the worker is the instrumentation that doesn't perturb the system.

### 8.1.5 React / Next.js at scale (be ready for one level)

Next.js: server-side rendering / static generation for landing and content pages (SEO, fast first paint), client-side hydration; on TVs, heavy hydration and large bundles hurt — code-splitting, lazy-loading the player, minimizing re-renders during playback (the player state shouldn't re-render the whole tree on every `timeupdate`). Living Room UIs need **spatial (D-pad) navigation** and memory discipline (TV browsers leak and crash under memory pressure).

**Self-test:** explain ABR in four sentences; name four Shaka components and their jobs; give two mechanisms by which a timestamp bug produces repeated video; explain why capability APIs aren't enough and how a decoder blocklist works with a TTL; explain why telemetry on the main thread can *cause* stalls.

---

## 8.2 MakeMyTrip (Senior SWE, Jun 2022 – Jan 2023)

Bullets: "Delivered 3+ production workflows across loans, credit, and foreign exchange …; cut application drop-off ~25% and API latency ~30%"; "Integrated 5+ lending, credit-scoring, and forex APIs with standardized retry, validation, and error handling (transaction success +18%); embedded KYC and RBI foreign-exchange controls into workflow logic, reducing manual review ~40%."

### 8.2.1 What KYC concretely requires from a software workflow (India)

Under the RBI's **Master Direction – Know Your Customer (KYC) Direction, 2016** (amended many times), a regulated entity must perform **Customer Due Diligence** before establishing a relationship:

- **Identity + address proof:** PAN (or Form 60) plus an **Officially Valid Document** (passport, driving licence, voter ID, proof of Aadhaar possession, etc.).
- **Verification modes:** Aadhaar **e-KYC** (OTP or biometric authentication via UIDAI, through licensed entities — OTP-based non-face-to-face e-KYC carries restrictions); **Central KYC Registry (CKYCR, run by CERSAI)** — fetch an existing KYC record by its identifier instead of re-collecting documents; **V-CIP** (video-based customer identification, permitted since 2020); digital document verification (DigiLocker).
- **Risk categorization** (low/medium/high) driving **periodic re-KYC**; **PEP and sanctions screening**; record retention (years after relationship ends); reporting suspicious transactions.
- **Aadhaar handling:** store only masked/tokenized Aadhaar numbers (Aadhaar Data Vault requirements) — a direct PII-engineering constraint.

**As software:** a KYC **state machine** per customer/application (`started → pan_verified → ovd_collected → ekyc_done | ckyc_fetched | vcip_scheduled → screening_passed → approved | manual_review | rejected`), each transition driven by a partner API result and a rule, every step audited. "Embedded controls into workflow logic" = the rules (document validity, name match thresholds between PAN and bank account, age, risk category) run **in code at each transition** instead of a human checking a queue — and only genuinely ambiguous cases route to manual review. That is **compliance-as-code**, the same family as your agent policy gates (QB §7).

### 8.2.2 What RBI foreign-exchange controls require

Outward remittances and forex purchases by resident individuals fall under **FEMA** and the RBI's **Liberalised Remittance Scheme (LRS)**:

- **Limit: USD 250,000 per resident individual per financial year** (April–March), aggregated across all banks/dealers **by PAN**; unused limit doesn't carry over.
- **PAN mandatory** for LRS transactions; **Form A2** declaration (amount + **purpose code**) for remittances; some purposes are prohibited.
- Only **Authorised Dealers** (banks, AD Category I/II) and **Full-Fledged Money Changers** can sell forex; a travel platform acts through them.
- **TCS (Tax Collected at Source)** on LRS remittances above a threshold, at purpose-dependent rates — the thresholds and rates **changed several times (2020, 2023, 2025)**, which is itself the engineering lesson: rate tables and limits must be **versioned configuration, effective-dated**, not code constants.

**As software:** before quoting/booking: validate PAN; compute the customer's **remaining LRS headroom** for the FY (partner/AD data + own records); validate purpose code against allowed purposes and required documents (e.g., passport/visa/ticket for travel forex); compute TCS by effective-dated rule; generate the A2 declaration; block or route to review on breaches. Each check cites the rule version that fired (audit).

**Also during your tenure — RBI Digital Lending Guidelines (Sept 2, 2022):** loans disbursed directly to the borrower's bank account and repaid from it (no pass-through/pool accounts); a **Key Fact Statement** with an all-inclusive **APR** before contract; a **cooling-off period** (≥ 3 days for tenor ≥ 7 days); LSP fees paid by the regulated entity, not charged to the borrower; need-based data collection with explicit consent. A travel platform offering loans is typically a **Lending Service Provider** partnering with a regulated lender. **PENDING:** whether your workflows implemented any of these (KFS display, consent capture) — if yes, it's a strong, dated, concrete compliance-as-code example.

#### 8.2.3 "Standardized retry, validation, and error handling across 5+ financial APIs"

**The idea:** each partner (lenders, credit bureaus, forex ADs, KYC providers) has its own error codes, timeouts, idempotency support and failure modes. Five bespoke integrations = five inconsistent behaviors. A **standard integration layer** gives every partner call the same semantics:

1. **Timeouts per partner**, from their SLA, plus an overall request deadline (budget propagated).
2. **Error taxonomy** mapping partner codes into **retryable** (timeouts on reads, 429, 502/503), **terminal** (validation errors, rejections), **user-fixable** (name mismatch, invalid document) → retry, fail, or ask the user.
3. **Retry policy:** exponential backoff + **jitter**, capped attempts, **only for idempotent operations** or operations carrying an **idempotency key**.
4. **Idempotency for payment-adjacent calls:** create-application / disburse / book-forex calls carry a client-generated idempotency key (or a stable reference id) derived from the *logical action*, stored *before* the first attempt (POST §13). **On a timeout of a non-idempotent call, never blindly retry — first do a status inquiry** ("did my booking with reference X go through?") and reconcile. **Credit bureaus are the sharp example:** a duplicate hard inquiry can affect the customer's credit score — idempotency here protects the user, not just the ledger.
5. **Circuit breaker per partner** — stop hammering a failing lender; fail fast or route to an alternate partner; half-open probes.
6. **Validation at both edges:** inputs before sending (PAN format `[A-Z]{5}[0-9]{4}[A-Z]`, IFSC, pincode, amounts within product limits) so partners don't reject avoidable requests; responses against a schema so a partner's malformed reply fails loudly at the boundary (an anti-corruption layer).
7. **Async callbacks/webhooks:** verify signatures, **dedupe on event id**, process out-of-order updates against a state machine (ignore a "pending" after "approved").
8. **Reconciliation jobs:** periodically compare your state with partner state for anything stuck in an intermediate status.
9. **PII handling:** encrypt at rest, mask in logs (PAN/Aadhaar/account numbers), tokenization, consent records, least-privilege access to raw documents, retention limits.
10. **Observability:** a correlation id per application flowing through every partner call — the same `traceId` discipline.

**How the numbers plausibly connect (PENDING — confirm which were true): +18% transaction success** — retries absorbing transient partner failures + pre-validation preventing avoidable rejections + status-inquiry reconciliation recovering "unknown" outcomes that used to be marked failed. **–30% API latency** — parallelizing independent partner calls (bureau pull alongside document checks), timeouts cutting the tail, caching static reference data. **–25% drop-off** — fewer failures and faster steps in the funnel; pre-filled data from CKYC. **–40% manual review** — rules-as-code auto-clearing unambiguous cases. Know each numerator/denominator and window.

**The bridge to agents (say it):** "An agent's tool layer is the same problem: external calls that fail transiently, non-idempotent writes, partial success, and the need to classify errors as retryable vs. terminal — I'd built that discipline for lending APIs before I built it for agent tools. 'Don't retry a 401' in my on-call story is the same rule."

**Self-test:** walk a forex booking through every compliance check in order; why must a timed-out disbursal not be retried blindly; why is a duplicate bureau pull worse than a duplicate read; what's in your error taxonomy; how would you make TCS rules change-safe.

### 8.3 AutoRABIT (Software Developer, Sep 2020 – Jun 2022)

Bullet: "Built reusable platform components, internal dashboards, developer tooling, and automated test infrastructure inside Salesforce release-management and CI/CD workflows, improving release reliability ~30%." AutoRABIT builds DevOps / release-management tooling (its ARM product) for Salesforce teams. **Why it matters here:** you have built **CI/CD developer tooling** — GitLab's core product — and you know a deployment model (Salesforce metadata) that is very different from "build an image, roll it out."

#### 8.3.1 Why Salesforce release management is not generic CI/CD

In a typical service, you build an immutable artifact and replace the running version. In Salesforce, **there is no artifact you replace**: the "application" is **metadata inside a long-lived, stateful org** (Apex classes and triggers, Lightning Web Components, custom objects and fields, flows, validation rules, page layouts, profiles, permission sets, sharing rules) plus the org's **data**. A deployment *mutates* the target org's configuration in place. Consequences:

- **Deployment unit = a metadata package**, described by `package.xml` (types + members), deployed via the **Metadata API** (`deploy()` of a zip; asynchronous — you poll status) or the Salesforce CLI (`sf project deploy start`, source-format projects from SFDX), or **change sets** (UI-driven, org-to-org).
- **Environments = sandboxes:** Developer and Developer Pro (metadata only; refreshable daily), Partial Copy (metadata + sample data; every 5 days), Full (complete copy; every 29 days). Promotion path: dev → integration/QA → UAT → staging (full copy) → production. **Refreshing a sandbox wipes it** — environment drift between sandboxes and prod is a constant problem.
- **Production gates enforced by the platform:** deploying Apex to production requires Apex tests to run (test levels: `NoTestRun` — not allowed for prod with Apex, `RunSpecifiedTests`, `RunLocalTests`, `RunAllTestsInOrg`) and **≥ 75% org-wide Apex code coverage**, with every trigger covered. Deployments are **all-or-nothing** in production (any failure rolls the whole deploy back).
- **Validate, then quick deploy:** a `checkOnly` validation runs all tests against prod without committing; a successful validation can be **quick-deployed** within a window (10 days) without re-running tests — this is how teams shrink the production change window.
- **Destructive changes:** deletions go in `destructiveChanges.xml` (and `destructiveChangesPre/ Post.xml` to control whether deletes happen before or after additions). Deleting a field deletes its data → destructive changes are the riskiest part of a release and deserve their own review.
- **No native rollback.** The platform rolls back a *failed* deploy, but once a deploy succeeds there's no "undo." Rollback = **redeploy the previous version of the metadata**, which only works if you **snapshotted the target org's metadata before deploying** — and data changes, deleted fields, and some component types can't be restored that way. Release tools therefore take a pre-deploy backup of affected components and generate a reverse package.
- **Merge pain is XML:** profiles and permission sets are huge, order-sensitive XML files; concurrent edits conflict constantly; **profile deployment semantics are partial** (a profile deploys only permissions for components included in the same package — a classic surprise).
- **Dependency ordering:** a field referenced by a flow or Apex must exist first; missing dependencies fail the deploy late.
- **Quality gates:** static analysis for Apex (PMD), Apex unit tests, UI tests, data seeding and masking for sandboxes.

#### 8.3.2 What "release reliability +30%" plausibly means (PENDING — OCQ B8)

Definition candidates: % of release deployments that succeed on first attempt; reduction in failed deployments / rollbacks / hotfixes per release; fewer production incidents traced to a release. Mechanisms your bullet names: **reusable components** (standard pipeline steps — validate, run tests, compute delta package, back up target), **automated test infrastructure** (running Apex and UI tests automatically on every merge instead of manually before

release), **dashboards** (deployment status, coverage, failures by component — visibility that catches problems before release night), **developer tooling** (delta-package generation from git diff, conflict detection on metadata). **Say exactly which you built, and how reliability was measured, or label it approximate.**

#### 8.3.3 Bridge to GitLab and to agents

- A **validate-then-quick-deploy** split is the same idea as "plan approved, then deterministic execution of exactly that plan."
- **Pre-deploy snapshot + reverse package** = rollback for mutable state — the same "state is the hard part of rollback" point as DB migrations and paused agent runs (Part 4.4).
- Agentforce v2 executes **config changes to live orgs** through Metadata/Tooling APIs — the same mutation model you tooled at AutoRABIT, now driven by an agent; it's why "access-management workflows" are lifecycle changes to a live system (DD §2.2 S1).
- For GitLab: pipelines, environments, protected environments with approvals, deployment tracking and rollback are product features; you've built the Salesforce-specific version of each.

**Self-test:** explain to a GitLab engineer why a Salesforce deploy can't be rolled back like a container image; name the four sandbox types and why refresh cadence matters; what does quick deploy buy; why are profiles painful in version control; what's in `destructiveChanges.xml` and why it's dangerous.

---

## Part 9 — Every resume number, and every resume skill, mapped

### 9.1 Every number, one table

Template for each (DD §2.1): **unit, baseline, method, attribution, what would move it.** "A number you can't derive the method for is a liability."

| # | Claim | What it means / arithmetic | Method status | Defended in |
|---|---|---|---|---|
| N1 | ~6 years production software; 2 years leading agentic SDLC at Salesforce | Sep 2020 → Oct 2026 ≈ 6 yrs; Salesforce Dec 2024 → now ≈ 1 yr 10 mo — say "nearly two years" | READY | — |
| N2 | ~60 days → ~5 days (~92%), five workflows | (60–5)/60 = 91.7%; most of 60 days = queue wait; remaining ~5 days mostly deliberate approval latency | **PENDING** median vs mean, n, window | DD §2.2 S1, Story A |
| N3 | Adopted across 10+ engineering teams | Adopters of integration guidance/standards | **PENDING** the traceable list | PROBE §13 |
| N4 | Mentors 5+ engineers | Design/code reviews; one engineer's concrete growth | **PENDING** one named example | DD §3.6, §10.2 |
| N5 | Eight capabilities onboarded to Headless 360 | API exposure + tool contract + Skill + permission mapping + evals each | **PENDING** list all eight + architecture each runs on | PROBE §1, Part 1.2.6 |
| N6 | Slack agent: 150+ users, 8 teams | Users of the agent | **PENDING** source | DD §2.2 S3 |
| N7 | ~20 min → under 3 min | ≥ 85% reduction; 3 min likely dominated by human approval | **PENDING** median vs p90 | DD §2.2 S3 |
| N8 | ~35% fewer misconfigured access requests | **Relative** reduction of (requests reaching execution/approval with invalid config ÷ all requests) | **PENDING** absolute before/after | DD §2.2 S5, Part 1.5 |

| # | Claim | What it means / arithmetic | Method status | Defended in |
|---|---|---|---|---|
| N 9 | Throughput ~40% up, stronger fault isolation | Unit candidate: completed workflow executions/hour at comparable load + resources | **PENDING** (OCQ F1–F5) | Part 4.2.5, DD §14.4 |
| N 10 | Hotstar player failure rate ~70% down | fatal-error sessions ÷ sessions started? which devices/window | **PENDING** | Part 8.1.3 |
| N 11 | MMT drop-off ~25%, API latency ~30% | Funnel completion; p50 or p95 latency? | **PENDING** | Part 8.2.3 |
| N 12 | MMT transaction success +18%, manual review ~40% | Relative or pp? numerator/denominator | **PENDING** | Part 8.2.3 |
| N 13 | AutoRABIT release reliability ~30% | First-attempt deploy success? rollbacks per release? | **PENDING** (OCQ B8) | Part 8.3.2 |
| N 14 | ResolveFlow ~2,750 chunks | 600 issues × ~4.6 chunks | **READY** | Part 2.2.6 |
| N 15 | Relevance floor 0.35 | Between 0.21–0.26 noise and 0.53–0.66 signal; midpoint 0.375/0.405 | **READY** (fix R11 comment) | Story E |
| N 16 | 50 scenarios, 86% routing accuracy, 24% lower cost | Wilson 95% for 43/50 ≈ [74%, 93%] | **HIGHEST RISK — PENDING** (R8) | Part 2.4, DD §4.7.6 |
| N 17 | Bug Triage ~90 → ~15 min | Diagnostic-context collection time, not fix time | **PENDING** n, who timed | DD §2.5 |
| N 18 | AdaptiveRAG +12% groundedness, –21% tokens, –18% median latency, 100 Qs | Relative vs pp; judge; paired test | **PENDING** | Part 2.1.8 |
| N 19 | Safety-gate suite | 11 tasks (README says 12 — fix) | **READY** after R9 fix | Part 3.1 |
| N 20 | `DAILY_ANALYZE_LIMIT` | Code says 30; CHANGELOG says 10 | **PENDING** align (R10) | — |

## 9.2 Every resume skill line → where it's taught

| Resume skill | Section |
|---|---|
| Autonomous agents for engineering task automation | 1.1, 7, DD §1.5–1.7 |
| Agent orchestration patterns | 1.1, 1.3 |
| Safety guardrails | 1.5, 3.3 |
| Tool/function calling; structured tool contracts | 1.2.1, 1.2.3, 2.3.4 |
| HITL approval workflows | 1.4 |
| Durable pause/resume (LangGraph interrupts) | 1.1.5–1.1.6 |
| Scoped write execution (OAuth-authorized) | 0.3 bridge 1, 7.2, F4 |
| CI signal correlation | 7 (check-runs on linked PR head SHA), 5.5 four signals |
| Code-review & investigation automation; Claude Code | 3.4, 5.6 (Bug Triage) |
| LangGraph, LangChain | 1.1 |
| MCP | 1.2 |
| Multi-agent systems | 1.3, 1.1.9 |
| Memory & state management | 1.1.5 (checkpointer vs Store), SG-GL §4 (compaction — HONEST GAP) |
| Model routing; model selection & token economics | 2.4 |

| Resume skill | Section |
|---|---|
| LLMs; RAG; adaptive & corrective RAG | 2.1 |
| Semantic search; vector retrieval strategies | 2.2, 2.3 |
| Reranking; chunking & retrieval optimization; embeddings | 2.3 |
| Structured output extraction | 2.3.4 |
| Prompt engineering | 2.4.3 (cache-friendly prompt layout), ResolveFlow prompt (2.1.4 / 7), 3.2 |
| OpenAI / Anthropic / open-weight APIs | 2.4.4 |
| Pinecone, ChromaDB, FAISS | 2.2 |
| AI evaluation frameworks; regression / retrieval / routing evaluation | 3.1 |
| Latency & cost analysis | 2.4.3, DD §5.6 |
| Groundedness & hallucination reduction | 3.2 |
| Model performance monitoring | 5.5 (online evals as SLIs), DD §4.8 |
| Guardrails; prompt-injection defense; input validation; output sanitization | 3.3 |
| Permission-aware execution; data governance | 1.5, 3.3 |
| Python, Go, TypeScript, JavaScript, Java, Apex, SQL | Python: ResolveFlow; Go: DD §4.13f; TS/JS: 8.1, Slack agent; Java: 4.3.1; Apex: 8.3; SQL: DD §4.13e, 7.2 |
| CI/CD (GitHub Actions, Jenkins); Git workflows | 4.4, DD §14.3 |
| Release management & deployment automation | 8.3, 4.4 |
| Testing frameworks | 3.4 |
| Observability (Splunk); performance monitoring | 5 |
| Documentation & reusable libraries/playbooks | DD §10.2, POST |
| AWS (Lambda, API Gateway, ECS) | 4.1 |
| Docker, Kubernetes | 4.2 |
| Spring Boot, REST, GraphQL, microservices | 4.3 |
| Distributed & event-driven systems; system & API design | 4.3.3, DD §6, DD §8 |
| Enterprise architecture patterns | 4.3.3 (strangler fig, ACL, outbox, saga) |
| Leadership: reviews, RCA, mentoring, cross-functional | 3.4, DD §3.3, §10 |
| Ruby (in JD, not on resume) | DD §11 |

---

# Part 10 — Master tiered question bank

Every question has a one-to-three-line answer sketch and a pointer. Status flags where your answer depends on unverified specifics. Practice out loud, no notes; anything you hesitate on goes on `GAPS.md` (WP).

## 10.1 Foundational

### Agents & orchestration

1. *What's the difference between an agent loop and a LangGraph graph?* — Loop: model chooses tools turn by turn, reason/act boundary is behavioral. Graph: legal transitions are code, state typed, every step checkpointed. (1.1.1)

2. *What is a `thread_id`?* — The key under which a run's checkpoints are stored; reuse it to resume. (1.1.5)
3. *What's a reducer, and when do you need one?* — How concurrent/sequential writes to a key combine; needed when parallel nodes write the same key (else `InvalidUpdateError`). (1.1.3)
4. *What does `interrupt()` do?* — Raises a control exception, checkpoints, returns the payload; resume with `Command(resume=x)`; node re-runs from the top. (1.1.6)
5. *Why is LangChain's `AgentExecutor` not what you'd use today?* — Opaque loop, no first-class durable pause or custom topology; moved to `langchain-classic` in 1.0; `create_agent` is built on LangGraph. (1.1.2)
6. *Function calling vs. MCP?* — Model-API feature to emit a structured call vs. a client-server protocol to discover and invoke tools living elsewhere. (1.2.1)
7. *Name MCP's server primitives and who controls each.* — Tools (model), resources (app), prompts (user). (1.2.3)
8. *Grounding vs. permission?* — Is it real vs. is this caller allowed; one check can't catch both. (1.5)

**Retrieval**

9. *What's an embedding?* — A vector where semantic similarity ≈ geometric closeness; only relative geometry matters. (2.3.1)
10. *Cosine vs. dot product vs. L2?* — Cosine = angle; dot = cosine for normalized vectors; L2 = distance. (2.2.2)
11. *Bi-encoder vs. cross-encoder?* — Separate encodings (fast, recall) vs. joint (slow, precision). (2.3.2)
12. *Why chunk, and why overlap?* — Fit context, keep one meaning per vector; overlap rescues boundary facts. (2.3.3)
13. *FAISS vs. Pinecone in one line each.* — Library in your process vs. managed vector DB. (2.2.1)
14. *What's recall@k vs. precision@k?* — Fraction of relevant found in top k vs. fraction of top k that's relevant. (RAG-S §6)

**Evaluation & safety**

15. *pass@k vs. pass^k?* — At least one succeeds (capability) vs. all succeed (safety). (3.1)
16. *Direct vs. indirect prompt injection?* — User-typed vs. planted in content the agent reads. (3.3)
17. *Hard vs. soft guardrail?* — Can the model talk its way past it? (1.5)

**Infra**

18. *Container vs. VM?* — Process with namespaces + cgroups sharing the kernel vs. full guest OS. (4.2.1)
19. *Pod, Deployment, Service?* — Unit of scheduling; desired-state manager of replicas; stable address in front of ready pods. (4.2.2)
20. *Liveness vs. readiness?* — Restart me vs. stop sending me work. (4.2.3)
21. *What is API Gateway, and is it compute?* — Front door: routing, auth, throttling; not compute. (4.1.3)
22. *Lambda max duration and the cold-start concept.* — 15 min; Init phase on a new environment. (4.1.2)
23. *REST idempotency of verbs.* — GET/PUT/DELETE idempotent; POST not → idempotency keys. (4.3.2)
24. *What does Spring Boot add over Spring?* — Auto-config, starters, embedded server, externalized config, Actuator. (4.3.1)

**Observability**

25. *Trace vs. span vs. log vs. metric?* (5.2, 5.5)
26. *What's in a `traceparent` header?* — version-traceid-spanid-flags. (5.3)

**Past work**

27. *What is Shaka Player?* — Google's OSS JS player for DASH/HLS over MSE/EME. (8.1.1)
28. *What is a Web Worker?* — Separate JS thread, no DOM, postMessage. (8.1.4)
29. *What does KYC require?* — CDD: PAN/Form 60 + OVD, e-KYC/CKYC/V-CIP, screening, risk-based re-KYC. (8.2.1)

30. *What is a Salesforce sandbox?* — A copy of the org's metadata (± data) for dev/test; four types with different refresh intervals. (8.3.1)

## 10.2 Senior

### Agents

1. *Walk me from a user request to a real side effect in your system.* — ResolveFlow superstep table (1.1.10) or Agentforce CM → Knowledge → Reasoner → Executor (6.2).
2. *What runs twice in your graph, and what breaks if an LLM call is moved above `interrupt()`?* — `_build_proposed_action`; approved text ≠ posted text. Hash-bind. (1.1.6, F2)
3. *Two approval clicks arrive together. What happens?* — Both load the paused checkpoint, both execute → two comments. Lock/lease + idempotency key. (F1)
4. *Why is your router rule-based?* — It allocates autonomy; must be cheap, deterministic; evolution = LLM proposer + rule floor. (7.2)
5. *How do you keep tool selection accurate with hundreds of tools?* — The funnel; per-step vector search → 3–5 → one grounded compile call; measure Stage A recall. (6.4)
6. *MCP: what's the lifecycle of a tool call?* — 2025-11-25: initialize → initialized → tools/list → tools/call; 2026-07-28: stateless, version + capabilities in `_meta`, `server/discover`. (1.2.4)
7. *How do you approve an artifact rather than an intention?* — Show the exact diff/plan, bind the approval to its hash, expire it, re-validate at resume. (1.4)
8. *When would you use `Send`?* — Fan-out of independent items in one superstep; reducers merge. (1.1.4)
9. *Subgraph with shared keys vs. called inside a node?* — Shared state vs. private schema mapped in/out. (1.1.7)

### Retrieval

10. *Explain CRAG precisely.* — T5-large evaluator; Correct/Incorrect/Ambiguous; strip-level decompose-then-recompose; keyword-rewritten web search. (2.1.4)
11. *Explain Adaptive-RAG precisely.* — Classifier on query complexity; no / single-step / multi-step; auto-labeled training. (2.1.6)
12. *Why did adaptive RAG reduce tokens and median latency in your platform?* — Easy queries skip retrieval/loops; tail may grow. (2.1.8) **PENDING** specifics.
13. *Your relevance floor — how was it chosen, and what's wrong with how?* — Two measured clusters; few points; PR curve on labeled pairs; coupled to embedding model. (Story E)
14. *When does reranking not help?* — Stage-1 recall miss; small clean candidate set. (2.3.2)
15. *HNSW knobs?* — `M`, `efConstruction` at build; `efSearch` per query. (2.2.2)
16. *How do you enforce tenant isolation in a vector store?* — Namespace / pre-filter / structural token; never post-filter. (2.2.5)

### Cost & models

17. *Why does an 8-turn agent loop cost superlinearly?* — Transcript resent each turn. (2.4.3)
18. *How do you lay out a prompt for caching?* — Tools → system → static examples → volatile last; deterministic tool order. (2.4.3)
19. *When is routing to a small model safe?* — When the node's eval holds within a pre-declared margin on a paired test. (2.4.2)

### Eval

20. *86% on 50 — what's the interval?* — Wilson ≈ [74%, 93%]. (3.1) **R8 PENDING**
21. *Compare two models on the same tasks — which test?* — McNemar on discordant pairs. (3.1)
22. *How do you evaluate a test-generation agent?* — Pass on head, fail on base/mutants, non-flaky over k runs; mutation score Δ. (DD §1.6)

23. *Groundedness vs. entailment — which does ResolveFlow check?* — Membership + relevance floor, not entailment (F5). (3.2)

**Infra**

24. *Design the AWS deployment of a durable agent service.* — API GW/ALB → admission Lambda (202) → SQS → ECS workers → RDS Postgres checkpoints; resume via enqueue. (4.1.5)
25. *Why might liveness probes cause an outage for agent workers?* — Dependency checks in liveness → restart storm killing in-flight steps. (4.2.3)
26. *Your Dockerfile — what would you change?* — Exec form / tini, non-root, multi-stage, and note Render doesn't run it. (4.2.1)
27. *Graceful shutdown for an agent worker on K8s?* — preStop drain, SIGTERM → stop pulling, checkpoint, release lease, exit within grace period. (4.2.3)
28. *Explain "throughput up 40%."* — Role first; mechanism (bulkheading?); unit; window. **PENDING** (4.2.5)
29. *REST or GraphQL for a new internal API between two services?* — Usually REST/gRPC; GraphQL for many heterogeneous clients over a connected domain. (4.3.2)
30. *How do you keep two microservices consistent?* — Saga + outbox + idempotent consumers. (4.3.3)

**Observability**

31. *How do you propagate trace context across a queue?* — `traceparent` in message headers; PRODUCER/CONSUMER spans; links for batches. (5.3)
32. *Where do Python traces break?* — Bare thread pools don't copy contextvars. (5.3)
33. *Show me the query you'd run to find why one tenant's run failed.* — Filter by `tenantId` + flow + `policy.decision`, follow `traceId`. (DD §0.2, 5.5)

**Past work**

34. *Walk through the Shaka timestamp bug.* — Timestamp → timeline mapping; re-fetch/re-append → repetition; your exact fix. **PENDING** your PR. (8.1.2)
35. *How did the decoder blocklist work, and what were its failure modes?* — Detect by device class → in-session switch → persist with TTL → fleet config; false positives degrade quality. (8.1.3)
36. *Why must a timed-out disbursal call not be retried?* — Non-idempotent; status inquiry first; idempotency key. (8.2.3)
37. *Why can't you roll back a Salesforce deploy like a container?* — Mutable org state; redeploy snapshot; data loss on destructive changes. (8.3.1)

## 10.3 Staff

1. *300 pipelines fail from one bad shared CI template and your fix-CI flow is on for all — what happens and what should?* — Amplification; fingerprint + collapse at admission; leader run; per-tenant buckets. (DD §5.2)
2. *Design the policy gate for an agent's diff. What can't it catch?* — Path/dep/test-deletion/ size rules → allow/ask/deny; can't catch semantically wrong but in-policy code → CI + review + revert tracking. (DD §4.13a)
3. *Two agents open MRs editing the same file.* — Plan-time overlap detection, CAS on commit (`last_commit_id`), ref CAS, merge train, rebase + re-verify. (DD §4.9, §6.6)
4. *Roll out a new prompt version across 10k projects safely.* — Flag per flow version → dogfood → % cohorts with online eval metrics vs. control; auto-halt on safety regressions; manifest on spans. (DD §5.5)
5. *Online resolve rate dropped 6 points overnight.* — FACET by flowVersion/modelId/promptVersion; then tenant/language mix; diff trajectories; patch the eval gap. (DD §9.2)
6. *Rebuild Agentforce's Supervisor/Specialists in LangGraph.* — CM supervisor graph; specialists as subgraphs returning via `Command.PARENT`; `GatherMoreDetails` as `interrupt`; Stage A as `Send`; Executor model-free with the only write credential. (1.1.9)

7. *Design an MCP gateway for external agents acting on GitLab.* — OAuth audience-bound tokens, no passthrough, per-tool approval modes, per-client allowlists, rate limits, pinned/hashed tool definitions, outputs untrusted, attribution, stateless revision for horizontal scale. (1.2.7, DD §8.8)
8. *Your corpus doubles with near-duplicate issues. What happens to your relevance floor?* — Score distributions compress; recalibrate; dedupe at ingest; consider reranker scores as the gate instead. (2.2.6)
9. *Bespoke tools vs. generic CRUD tools — when, and how does the security analysis change?* — Bespoke: narrow blast radius, doesn't scale; generic: scales via NLDs but needs entity/field allowlists, value constraints, platform FLS underneath. (AS §1.5)
10. *Pick a vector store for GitLab self-managed customers.* — Likely Postgres/pgvector (already operated, same ACLs/transactions, no egress); FAISS for embedded read-only cases; managed SaaS only where allowed. Reasoning, not a claim about GitLab. (2.2.4)
11. *Design the cost model for an agent platform; what's the unit?* — Cost per successful task; p50/p95 cost per run; drivers ranked; caching and routing levers; per-tenant budgets. (2.4.3)
12. *Model provider degraded: p95 4×, 8% 5xx.* — Circuit breaker; interactive → evaluated fallback model; background → checkpoint + retry later; tag `fallback=true`. (DD §9.7)
13. *Lambda vs. ECS for the agent core — defend your choice under 10× traffic.* (4.1.5)
14. *A trace breaks at the approval pause in production. How do you fix the observability?* — Persist trace context in the checkpoint; span link on resume; same correlation attributes. (5.3)
15. *How would you measure the decoder-blocklist's correctness at fleet scale?* — Per-entry hit counts, recovery after TTL expiry, failure rate by device class pre/post, quality cost (bitrate/resolution delta). (8.1.3)
16. *How do you make regulatory rules (TCS, LRS limits) change-safe?* — Effective-dated versioned config, rule id in the audit record, tests per rule version, backfill-safe evaluation. (8.2.2)
17. *Salesforce release pipeline: where do you put the gates?* — PMD + unit tests on MR; validate (checkOnly) against staging/prod; quick deploy in window; destructive changes as a separate reviewed step; pre-deploy snapshot for rollback. (8.3.1)
18. *Compaction: what must never be compacted in ResolveFlow, and where does it go?* — Original issue/evidence ids, current proposed action + its hash, approval state, thread owner → protected state fields or a Store. **HONEST GAP** (not built). (SG-GL §4)

### 10.4 Principal / Fellow (assumption removal, judgment, scale)

1. *Remove the human approval: what changes in ResolveFlow's safety argument?* — The remaining controls are the deterministic gates and the write scope; you'd need artifact-level policy, rate limits, lower-impact write set, and post-hoc audit; or keep humans for T3+. Walk the litmus test. (11.2)
2. *Composite identity with no human (alert-triggered run): whose authority?* — Project owner / on-call as the human half, or propose-only (draft MR, comment). (DD §1.10 #5, §7.1)
3. *An agent's token for project A is replayed against project B. Which control stops it?* — Audience/resource binding (RFC 8707 / `aud`), intersection with project membership, per-run minting at publish time. (1.2.5, DD §7.1)
4. *Design checkpoint storage for a million paused runs with daily schema changes.* — Partition, latest-hot/history-cold, TTL, versioned state + upgraders, re-mint creds at resume, leases. (DD §8.6)
5. *MCP went stateless in 2026. What does that change for a gateway design and for long-running tools?* — Any replica serves any request; cross-call state via server-minted handles; long work via the Tasks extension with polling and durable handles; trace context in `_meta`. (1.2.4)
6. *Where does LangGraph's durability model end and Temporal / Lambda durable functions begin?* — All are checkpoint + replay-with-memoized-steps; Temporal/durable functions give timers, signals, workflow versioning as first-class; LangGraph gives model-driven dynamic topology and HITL primitives; a production platform may use both. (1.1.5, 4.1.2, DD §2.2 S7)
7. *If you could build only three platform primitives for every agent team?* — Identity/credential broker with operation-bound tokens; durable run state machine with leases + checkpoints; eval + trace pipeline with the execution manifest. (DD §4.15)
8. *How should GitLab measure whether agents make customers' engineering better, not just faster?* — Merge × revert × human edit distance × cycle time vs. matched human baseline; DORA with an agent dimension. (DD

§4.15)

9. *Self-managed customers with their own models and no telemetry egress — eval/observability strategy? —* Shipped offline eval packs, model qualification suite, local metrics, opt-in aggregates; OTel lets them export to their own backend. (DD §4.15, 5.1)
10. *Kill-switch criteria for an agent flow GitLab-wide. —* Confirmed unsafe action; revert rate above human baseline with significance; cost runaway; any cross-tenant leak. (DD §4.15)
11. *You inherit a platform 10–20% slower than its predecessor, with an uncertain retrieval pipeline and parity gaps. Prioritize. —* Measure first (stage latency breakdown, eval regressions); fix correctness/safety, then the scaling risk with the largest blast radius (retrieval), then latency with parallel execution of independent steps; communicate the trade-off explicitly. (6.6, QB §9)
12. *Agents push at "agent rate." What breaks first and the cheapest intervention for each? —* CI capacity and human review attention before tokens; admission collapse, targeted tests, merge train lanes, review caps. (DD §5.1, §6)
13. *Your on-call story: tell it as a trace query. —* **PENDING** real incident (OCQ D6); rehearse the DD §14.5 shape meanwhile.
14. *What's the weakest claim on your resume, and what are you doing about it? —* Say R8 (or the 40% / Data Cloud role) before he finds it, with the remediation. This is a Fellow-grade answer. (7.4)

### 10.5 IAM-flavored drill (his specialty — practice separately)

1. Authentication vs. authorization in one sentence each, then where each happens in ResolveFlow.
2. OAuth authorization code flow with PKCE — each step, what `state` protects against.
3. Why server-side `state` in ResolveFlow (multi-domain aliases; cookie scope).
4. Delegation vs. impersonation; Azure AD on-behalf-of as the analogy for composite identity (analogy, not a claim about GitLab's implementation).
5. Token lifetime vs. a six-hour approval pause: re-mint at resume, re-check authorization.
6. Revocation mid-run: why minting at publish time (not admission) makes it fail safe.
7. RFC 8693 token exchange — your `orgJWT` example.
8. Audience binding — RFC 8707 resource indicators; JWT `aud`.
9. What goes in the audit record for an agent action (DD §7.5).
10. BOLA/IDOR — your `thread_owners` fix.

---

## Part 11 — One-page closing recap

### 11.1 The ten sentences to have verbatim

1. "The model proposes; deterministic code disposes — and the agent never holds authority of its own; it borrows the caller's, narrowed."
2. "A graph encodes the legal transitions as code; the only path to the side-effecting node is through the approval node, and that node re-checks anyway."
3. "Code before `interrupt()` runs twice; a checkpointer gives you history, not mutual exclusion."
4. "Grounding asks if it's real; permission asks if the caller's allowed — one check can't do both."
5. "Corrective RAG fixes bad retrieval after it happens; adaptive RAG avoids unnecessary or insufficient retrieval before it happens."
6. "Bi-encoders for recall, cross-encoders for precision."
7. "The cheapest model call is the one you don't make; then shrink the input; then route; then cache."
8. "Liveness says restart me; readiness says stop sending me work — dependencies never go in liveness."
9. "Propagate traceId, upstream trigger, tenant and user through every hop — including across the pause, by persisting the trace context in the checkpoint."
10. "A number without a method is a liability; a number without an interval is an anecdote."

### 11.2 The litmus test (DD §4.16)

> "If the model in this flow is fully controlled by an attacker who wrote the issue, what is the worst thing that can reach `main` — and how many independent controls would have to fail first?" If any layer's answer is "the prompt tells it not to," that layer is not a control.

### 11.3 Before-the-interview self-check (answer each out loud, Oct 8 evening)

- [ ] I can draw ResolveFlow's graph and trace one run superstep by superstep, including what runs twice. (1.1.10)
- [ ] I can draw Agentforce v2 from memory and map it to LangGraph primitives. (1.1.9, 6.2)
- [ ] I can explain MCP's primitives, both lifecycle revisions, and the no-passthrough rule. (1.2)
- [ ] I can define naive / reranked / CRAG / Self-RAG / Adaptive-RAG precisely and cite the papers. (2.1)
- [ ] I know whether AdaptiveRAG's +12% is relative or pp, how it was judged, and how FAISS loaded on Lambda. (2.1.8, 4.1.6) — **PENDING until done**
- [ ] R8 is resolved: committed, rebuilt, or I will raise it myself first. (7.4)
- [ ] R9, R10, R11 fixed in the repo. (7.4)
- [ ] Every infra row in 4.4 has an OWN / PATTERN / THIN rating and one boundary sentence. (OCQ)
- [ ] I have one real on-call incident with timestamps and the query I ran. (OCQ D6)
- [ ] I have found my Shaka Player PR and can state its root cause and fix. (8.1.2)
- [ ] I know the denominators for 92%, 35%, 40%, 70%, 18%, 30%. (9.1)
- [ ] I can do the Wilson interval for 43/50 and explain McNemar. (3.1)
- [ ] I have my three questions for him chosen. (DD §12)

### 11.4 The opening frame (DD §0.2b) and the close

**Open:** "*I build agents where the model proposes and deterministic code disposes, and where the agent never holds authority of its own — it borrows the caller's, narrowed. I've done it in production at Salesforce, in multi-tenant access-management automation, and end-to-end in public code: a LangGraph agent that diagnoses GitHub issues and writes back only after an independent check and a human approval, checkpointed in Postgres, running under the approver's own OAuth token.*"

**If stuck anywhere:** name the unknown, state the assumption you'll proceed under, say what experiment or metric would resolve it. **Never bluff.** On infra: role first, then one level deeper than he expects on the part you owned.

**First question to ask him:** "*You're a couple of months in — what does your team own today, and where's the biggest gap between the Duo Agent Platform vision and what's actually shipped?*"

---

### Appendix — Sources verified for the new sections (Oct 2, 2026)

- LangGraph interrupts and persistence: docs.langchain.com/oss/python/langgraph/interrupts, /persistence; durability modes (exit / async / sync) per LangGraph docs and release notes; LangChain v1 migration guide (AgentExecutor → `langchain-classic`; `create_agent` on LangGraph).
- MCP: modelcontextprotocol.io/specification/2025-11-25 and /2026-07-28 (+ changelog, versioning).
- CRAG: Yan et al., arXiv 2401.15884. Self-RAG: Asai et al., arXiv 2310.11511. Adaptive-RAG: Jeong et al., NAACL 2024, arXiv 2403.14403.
- Anthropic prompt caching: platform.claude.com/docs/en/build-with-claude/prompt-caching.
- AWS: API Gateway integration timeout increase (AWS What's New, June 4 2024); Lambda durable functions (re:Invent 2025, AWS Lambda docs).
- Shaka Player architecture tutorial (shaka-project.github.io); Shaka issues on HLS live looping / timestamp offsets; Shaka temporary variant disabling (`maxDisabledTime`, PR #4189).

- Disney+ Hotstar 5.9 crore peak concurrency (Inc42, Social Samosa, Nov 2023).
- RBI LRS (USD 250,000/FY, PAN, Form A2); RBI Digital Lending Guidelines (Sept 2, 2022).
- Chroma architecture (embedded / client-server, HNSW, Rust core).

**Marked as needs-verification rather than stated as fact:** your Shaka PR's exact root cause; your decoder-blocklist's exact mechanism; which KYC/forex/digital-lending controls your MMT workflows implemented; AutoRABIT reliability metric definition; AdaptiveRAG implementation details; all Salesforce infra specifics (OCQ); the MCP revision Headless 360 used; OpenAI's current caching discount; exact current TCS thresholds (changed repeatedly — treat as config).

---